POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Different Approaches for Lift Coefficient Prediction on an Airfoil: Classical CFD and Deep Learning Techniques

Project for the course of Advanced Methods for Scientific Computing

Giulio Enzo Donninelli, Alessia Rigoni, Vittorio Sironi and Alessandro Verrengia

Advisor:
Prof. Luca Dedè

Academic year:
2025-2026

Abstract: This work presents an integrated computational pipeline for rapid aerodynamic analysis, bridging high-fidelity simulations with a data-driven surrogate model. We extended the `LifeX` C++ finite element library with a Reynolds-Averaged Navier-Stokes (RANS) solver using the Spalart-Allmaras turbulence model. This solver generated a reference dataset on the CINECA Leonardo HPC cluster, which was used to train a Convolutional Neural Network (CNN) built from scratch in C++ . The CNN predicts lift coefficients from a Signed Distance Function (SDF) geometry representation and is trained in parallel using MPI. The model achieves a validation Mean Squared Error of 0.0104 and a 35× speedup over direct simulation, providing a robust framework for efficient aerodynamic design.

1. Introduction

Aerodynamic design balances a trade-off between the demand for physical fidelity and the need for rapid design iteration. High-fidelity Computational Fluid Dynamics (CFD) simulations, such as those based on the Reynolds-Averaged Navier-Stokes (RANS) equations, offer unparalleled accuracy in predicting complex flow phenomena. However, their significant computational cost makes them impractical for design-space exploration or real-time analysis. Conversely, data-driven surrogate models, especially deep learning architectures like Convolutional Neural Networks (CNNs), can provide near-instantaneous predictions but are entirely dependent on the quality and availability of the high-fidelity data used for their training.

This project develops a pipeline that integrates both methods. Inspired by the framework of Cuozzo et al. (2026), we first build a high-performance CFD solver to generate reference-quality aerodynamic data. We then use this data to train a custom CNN surrogate model. By establishing this workflow for clean symmetric airfoils, we made a base idea for future extensions to more complex iced geometries.

The document is structured to follow this approach, beginning with the development of the high-fidelity simulation framework. Section 2 (**Navier-Stokes Simulation**) details the extension of the `LifeX` C++ finite element library to handle external aerodynamic flows. We describe the integration of the one-equation Spalart-Allmaras RANS model, the adaptation of the solver to two-dimensional domains, the implementation of specialized boundary conditions and the development of utilities for computing aerodynamic forces. This section also provides a comprehensive overview of the High-Performance Computing (HPC) workflow established on the CINECA Leonardo cluster.

Building upon the dataset generated by these simulations, Section 3 (**Convolutional Neural Network**), presents the design, implementation and training of our surrogate model. A key contribution of this work is

the development of a CNN entirely from scratch in C++ , designed to predict the lift coefficient from an airfoil's geometric representation along with the angle of attack and Reynolds number. We elaborate on the mathematical foundations of the network architecture, the physics-informed loss function that regularizes training and the implementation of distributed-memory parallelism using MPI, enabling efficient execution on HPC hardware.

2. Navier-Stokes Simulation

The main goal of this section of the project is to extend the `LifeX` library with new features, making it a more general-purpose and high-performance simulation framework. Specifically, the work focuses on integrating one-equation **RANS turbulence model** (Spalart-Allmaras) into the existing incompressible **Navier-Stokes solver** (NS), enabling the simulation of turbulent flows in external aerodynamic configurations.

2.1. `LifeX` Implementation

As detailed by Africa et al. (2024), `LifeX` is an open-source C++ library built on top of `deal.II`, originally designed for life-science applications such as cardiac and cardiovascular fluid dynamics. While the library provides a robust and well-structured finite element infrastructure, its original focus on biomedical applications necessitated several non-trivial adaptations to extend it to external aerodynamic flows.

This work addressed four primary challenges:

- adapting the solver and utility functions, originally designed for three-dimensional problems, to two-dimensional domains (Section 2.1.1);
- integrating the one-equation Spalart-Allmaras (SA) RANS model to account for turbulent effects (Section 2.1.2);
- implementing specialized conditions for aerodynamic simulations, including wall functions and far-field boundaries (Section 2.1.3)
- developing a residual-based post-processing utility to compute lift and drag forces (Section 2.1.4).

The following sections detail the implementation of each of these components.

2.1.1. 2D Implementation

While the `LifeX` `FluidDynamics` solver is templated on the spatial dimension `dim` through the `deal.II`convention, its original test cases and utilities were primarily designed for three-dimensional domains. Extending the framework to 2D involved adapting three key components: the finite element function space, the calculation of vorticity and the computation of the wall distance field.

Function space and triangulation

The SA solver uses Q_1 **finite elements** (`FE_Q` of degree 1) on the same triangulation as the NS solver:

$$\tilde{\nu}_h \in V_h = \left\{ v \in H^1(\Omega) \;\middle|\; v\big|_{\partial\Omega_D} = g_D \right\} \tag{1}$$

On the implementation side, the SA Degree of Freedom (DoF) handler (`dof_handler_sa_`) is independent from the NS one (`dof_handler`), in a way that the two systems have disjoint degrees of freedom without altering the block structure of the Navier-Stokes system. The shared triangulation object is the key architectural choice that allows both DoF handlers to be iterated in lockstep during matrix assembly.

Vorticity in 2D

The modified vorticity $\tilde{S}$ entering the SA production term requires the vorticity magnitude $S = |\boldsymbol{\omega}|$. In three dimensions, this is computed as $S = |\nabla \times \mathbf{u}|$.

In a 2D context, it simplifies to the absolute value of the scalar curl:

$$S = \left| \frac{\partial u_y}{\partial x} - \frac{\partial u_x}{\partial y} \right| \tag{2}$$

which is extracted from the velocity gradient tensor $\nabla \mathbf{u}^{n+1}$ interpolated at quadrature points via `fe_values_ns_.get_function_gradients`.

Wall distance in 2D

The wall distance $d(\mathbf{x})$ is computed once during `setup()` via a parallel algorithm. Each MPI rank collects the barycenters of boundary faces tagged as `prm_wall_tag_` (airfoil surface), serializes them into a flat `double` array of size $N_{\text{face,loc}} \times 2$, and participates in a global `MPI_Allgatherv` to build the complete set $\{\mathbf{c}_w\}_{w=1}^{N_{\text{face,tot}}}$.

The distance at every locally owned SA node i is then:

$$d_i = \max\Big(\min_w \|\mathbf{x}_i - \mathbf{c}_w\|_2,\ 10^{-12} \Big) \tag{3}$$

where the lower clamp prevents division by zero for nodes sitting exactly on the wall boundary.

The result is stored in `wall_distance_ghosted_`, a Trilinos ghosted vector indexed on `dof_handler_sa_`, following the standard pattern:

1. write into an owned vector and call `compress(VectorOperation::insert)`.
2. copy owned → ghosted to propagate ghost entries across MPI ranks.

The overall complexity per rank is $O(N_{\text{face,tot}} \cdot N_{\text{nodes,loc}})$, which is acceptable for the 2D meshes used in the airfoil example.

2.1.2. Turbulence Model

To enable the simulation of turbolent flows, the framework was extended with the one-equation SA RANS model, introduced by Spalart and Allmaras (1992). This model introduces a transport equation for a scalar variable known as the *modified kinematic viscosity*, $\tilde{\nu}$:

$$\boxed{\frac{\partial \tilde{\nu}}{\partial t} + \mathbf{u} \cdot \nabla \tilde{\nu} = \underbrace{c_{b1} \tilde{S} \tilde{\nu}}_{\text{production}} - \underbrace{c_{w1} f_w \left(\frac{\tilde{\nu}}{d} \right)^2}_{\text{destruction}} + \underbrace{\frac{1}{\sigma} \Big[\nabla \cdot \big((\nu + \tilde{\nu}) \nabla \tilde{\nu} \big) + c_{b2} |\nabla \tilde{\nu}|^2 \Big]}_{\text{diffusion + cross-diffusion}}} \tag{4}$$

The model constants (Table 1) follow the standard set by **NASA Turbulence Modeling Resource (TMR)** without the f_{t2} trip term.

Table 1: Spalart-Allmaras model constants based on the NASA TMR standard.

Constant	Value	Role
c_{b1}	0.1355	production coefficient
c_{b2}	0.622	cross-diffusion coefficient
σ	2/3	turbulent Prandtl number
κ	0.41	von Kármán constant
c_{v1}	7.1	viscous damping
c_{w1}	≈ 3.2391	destruction (derived from log-law consistency)
c_{w2}	0.3	regularization in f_w
c_{w3}	2.0	regularization in f_w

Closure functions

The model relies on a set of non-linear closure functions to relate $\tilde{\nu}$ to the turbulent eddy viscosity. Defining the viscosity ratio $\chi = \tilde{\nu}/\nu$, these functions are:

$$f_{v1}(\chi) = \frac{\chi^3}{\chi^3 + c_{v1}^3}, \qquad f_{v2}(\chi) = 1 - \frac{\chi}{1 + \chi f_{v1}(\chi)},$$

$$\tilde{S} = \max\left(S + \frac{\tilde{\nu}\, f_{v2}}{\kappa^2 d^2},\ 10^{-10} \right),$$

$$r = \min\left(\frac{\tilde{\nu}}{\tilde{S}\,\kappa^2 d^2},\ 10\right), \quad g = r + c_{w2}(r^6 - r), \quad f_w(g) = g\left(\frac{1 + c_{w3}^6}{g^6 + c_{w3}^6}\right)^{1/6},$$

The final turbulent eddy viscosity, ν_t, which modifies the momentum equations, is then computed as:

$$\nu_t = \tilde{\nu}\, f_{v1}(\chi).$$

Time discretization and operator splitting.

To solve the coupled Navier-Stokes and Spalart-Allmaras system, this is advanced at each time step using a **first-order Lie splitting** scheme (splitting error $O(\Delta t)$). This approach is consistent with the first-order BDF1 time integration used for both solvers.

Given the state $(\mathbf{u}^n, p^n, \tilde{\nu}^n)$ at time t^n, the solution at t^{n+1} is obtained in two steps:

1. the **NS step** solve the momentum and continuity equations with the with effective viscosity frozen at $\tilde{\nu}^n$:

$$\rho\frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} + \rho(\mathbf{u}^* \cdot \nabla)\mathbf{u}^{n+1} - \nabla \cdot \left(2\mu_{\text{eff}}^n \nabla^S \mathbf{u}^{n+1}\right) + \nabla p^{n+1} = \mathbf{0},$$

   where $\mu_{\text{eff}}^n = \mu + \nu_t(\tilde{\nu}^n)$ is *explicit* with respect to the SA variable;

2. the **SA step** solve the transport equation using the velocity field $\mathbf{u}^{n+1}$ just computed and a semi-implicit linearization:

$$\frac{\tilde{\nu}^{n+1} - \tilde{\nu}^n}{\Delta t} + \mathbf{u}^{n+1} \cdot \nabla\tilde{\nu}^{n+1} = c_{b1}\tilde{S}^n\tilde{\nu}^{n+1} - c_{w1}f_w^n\frac{\tilde{\nu}^n}{d^2}\tilde{\nu}^{n+1} + \frac{\nu + \tilde{\nu}^n}{\sigma}\Delta\tilde{\nu}^{n+1} + \frac{c_{b2}}{\sigma}|\nabla\tilde{\nu}^n|^2.$$

The linearization strategy is the following:

- the **production** is treated implicitly using a Picard iteration. $\tilde{S}^n$ is frozen and $\tilde{\nu}^{n+1}$ appears linearly in the matrix (stabilizing for $\tilde{S}^n > 0$);
- the **destruction**is linearized around $\tilde{\nu}^n$. The coefficient $c_{w1}f_w^n\tilde{\nu}^n/d^2$ enters as a positive reaction term, ensuring coercivity;
- about the **Diffusion**, the coefficient $(\nu + \tilde{\nu}^n)/\sigma$ is explicit while $\Delta\tilde{\nu}^{n+1}$ is implicit;
- about the **Cross-diffusion**, $c_{b2}/\sigma\,|\nabla\tilde{\nu}^n|^2$ is fully explicit and moved to the right-hand side.

Weak formulation and SUPG stabilization

To handle the advection-dominated nature of the SA equation, the weak form is stabilized using the **Streamline-Upwind Petrov-Galerkin** (SUPG) method.

Multiplying by the SUPG test function $\psi_h = v_h + \tau_{\text{SUPG}}\,\mathbf{u} \cdot \nabla v_h$ and integrating over Ω, the discrete linear system components become:

$$A_{ij}^K = \int_K \left[\psi_i\frac{\phi_j}{\Delta t} + \psi_i(\mathbf{u} \cdot \nabla\phi_j) + D\,\nabla\phi_i \cdot \nabla\phi_j - \psi_i\,c_{b1}\tilde{S}^n\,\phi_j + \psi_i\,c_{w1}f_w^n\frac{\tilde{\nu}^n}{d^2}\,\phi_j\right] dK,$$

$$b_i^K = \int_K \psi_i\left[\frac{\tilde{\nu}^n}{\Delta t} + \frac{c_{b2}}{\sigma}|\nabla\tilde{\nu}^n|^2\right] dK,$$

where $D = (\nu + \tilde{\nu}^n)/\sigma$.

The stabilization parameterτ_{SUPG} is defined locally for each cell K as:

$$\tau_{\text{SUPG}} = \frac{1}{\sqrt{\left(\frac{2}{\Delta t}\right)^2 + \left(\frac{2|\mathbf{u}|}{h_K}\right)^2 + \left(\frac{4D}{h_K^2}\right)^2}},$$

where h_K is the minimum vertex distance of cell K.

Physical bound enforcement.

After each linear solve, a physical bound is enforced on the owned vector before updating the ghosted copy (the ordering `compress(VectorOperation::insert)` → ghosted copy is mandatory):

- negative values ($\tilde{\nu} < 0$) are clamped to 0.0, since turbulent viscosity cannot be negative (SA physical bound);
- `NaN` values ($\tilde{\nu} = \texttt{NaN}$) are reset to $\tilde{\nu}_\infty$ (freestream value) rather than zero, so that the affected cell remains active and self-corrects over subsequent time steps, avoiding a permanently "dead" region in the domain.

Adaptive time-stepping and bisection.

The bisection mechanism of `FluidDynamics` is fully supported.

`try_timestep_with_bisection` is declared `virtual` in the base class, and `FluidDynamicsSA` overrides it by calling `sa_solver.take_snapshot()` before each attempt and `sa_solver.restore_snapshot()` on failure, ensuring that the SA state $\tilde{\nu}^n$ is rolled back consistently at every bisection level.

2.1.3. Boundary Conditions

The implementation handles three distinct boundary condition types (the airfoil wall, the farfield inlet and the outlet boundaries) imposed on the SA variable each with a different physical motivation.

Wall-resolved (low-Re regime).

A homogeneous Dirichlet condition is applied on the airfoil surface: $\partial\Omega_{\text{wall}}$:

$$\tilde{\nu} = 0 \quad \text{on } \partial\Omega_{\text{wall}}. \tag{5}$$

This is the physically correct condition in the viscous sublayer, as the viscosity ν_t must vanish as the distance to the wall $d \to 0$.

Wall functions (high-Re regime).

When the first mesh node is at $y^+ > 5$ (unresolved sublayer), a non-homogeneous Dirichlet condition is derived from the **log-law**:

1. the friction velocity is estimated via Schlichting's flat-plate formula:
$$C_f = \frac{0.026}{Re^{0.143}} \qquad \text{and} \qquad u_\tau = U_\infty \sqrt{C_f/2}$$
2. the wall-normal distance of the first node is converted to wall units:
$$y^+ = \frac{d_{\text{node}}\, u_\tau}{\nu}.$$
3. a switch criterion at $y^+_{\text{sw}} = 11.06$ separates the sublayer from the log-layer:
$$\tilde{\nu}_{\text{wall}} = \begin{cases} 0 & \text{if } y^+ < y^+_{\text{sw}}, \\ \text{root of } \nu_t(\tilde{\nu}) = \nu\kappa y^+ & \text{otherwise.} \end{cases}$$
4. the inversion of $\nu_t(\tilde{\nu}) = \tilde{\nu}\, f_{v1}(\tilde{\nu}/\nu)$ is performed with a **fixed-point iteration**:
$$\tilde{\nu}^{(k+1)} = \frac{\nu\kappa y^+}{f_{v1}\left(\tilde{\nu}^{(k)}/\nu\right)}, \quad k = 0, \ldots, 4,$$
which converges in 5 iterations due to the monotonicity of f_{v1}.

Inlet and farfield.

A Dirichlet condition with freestream value is imposed:

$$\tilde{\nu} = \tilde{\nu}_\infty \in [3\nu,\, 5\nu] \quad \text{on } \partial\Omega_{\text{inlet}} \cup \partial\Omega_{\text{farfield}}. \tag{6}$$

Outlet.

No condition is explicitly imposed: the natural Neumann condition $\partial_n \tilde{\nu} = 0$ arises automatically from the boundary term produced by integration by parts of the diffusion operator in the weak form.

2.1.4. Aerodynamic Coefficients (Residual-Based Approach)

Lift and drag coefficients are computed at the end of each time step via a **residual-based boundary force method**, using the `FluidDynamics::compute_force(tag)` method available in `LifeX`, which integrates the full stress tensor over the boundary face with the prescribed tag (airfoil wall):

$$\mathbf{F} = \int_{\partial\Omega_{\text{wall}}} \boldsymbol{\sigma} \cdot \mathbf{n} \, d\Gamma, \qquad \boldsymbol{\sigma} = -p\,\mathbf{I} + 2\mu_{\text{eff}}\,\nabla^S \mathbf{u}. \tag{7}$$

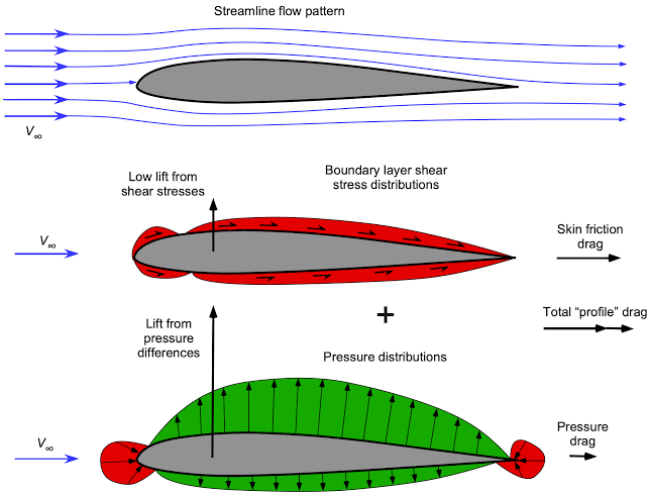


Figure 1: Aerodynamic forces on an airfoil. Source: Eagle Pubs

As illustrated in Figure 1, the Cartesian force components (F_x, F_y) are projected into the aerodynamic reference frame using the angle of attack α:

$$D = F_x \cos\alpha + F_y \sin\alpha, \qquad L = -F_x \sin\alpha + F_y \cos\alpha. \tag{8}$$

The forces are normalized by the dynamic pressure q_∞ and the reference surface S_{ref} (which corresponds to the chord length in 2D simulations):

$$q_\infty = \tfrac{1}{2}\rho U_\infty^2, \qquad C_L = \frac{L}{q_\infty S_{\text{ref}}}, \qquad C_D = \frac{D}{q_\infty S_{\text{ref}}}. \tag{9}$$

Implementation details

This procedure is encapsuled in a header-only utility class `AerodynamicCoefficientsEvaluator` that exposes a single method:

```
compute_coefficients(tag, density, U_inf, S_ref, alpha)
```

Listing 1: Method from `AerodynamicCoefficientsEvaluator`.

It is registered as a callback via `FluidDynamics::set_end_timestep_callback`, so it executes automatically after each successful NS time step. Results are then appended to a CSV file (`force.csv`) with columns `time, F_lift, F_drag, C_L, C_D`.

2.1.5. Configuration

The turbulence model is selected via the parameter `"Turbulence model": "Spalart-Allmaras"` in the JSON configuration file.

For backward compatibility, `FluidDynamicsSA` also accepts the legacy value `"None"`, overriding it internally with `Turbulence::SpalartAllmaras` after `parse_parameters`.

The `enum class Turbulence` in `fluid_dynamics.hpp` was extended accordingly:

```
enum class Turbulence { None, VMS_LES, SpalartAllmaras };
```

Listing 2: Extension of the class `Turbolence`.

2.2. Computational framework and execution workflow

This section details the computational environment and the end-to-end workflow established for the simulation.

2.2.1. HPC platform and software stack

All simulations were executed on **Leonardo** at CINECA (Bologna), targeting the `dcgp_usr_prod` partition (Intel Sapphire Rapids nodes, 112 cores per node) under the EuroHPC allocation `EUHPC_D35_025`, with job submission managed by the SLURM scheduler.

The simulation framework is `LifeX` 2.1, introduced in Section 2.1, compiled against `deal.II`9.7.1 with **Trilinos** as the linear-algebra backend, MPICH 4.0 for distributed-memory parallelism, and HDF5 1.14.5 for field output in XDMF format. Meshes are generated externally with Gmsh and read by `LifeX` in MSH 2.2 format.

The full software stack is delivered to the compute nodes through an Apptainer image (`lifex_env.sif`) maintained by the `LifeX` developers, which removes the need for any host-level `module load` of HPC libraries; the build procedure and the rationale behind this choice are detailed in Section 2.2.2.

The resources allocated for each run of the simulation campaign are summarized in Table 2.

Component	**Specification**
Cluster	Leonardo @ CINECA (EuroHPC)
Partition	`dcgp_usr_prod`
CPU	Intel Sapphire Rapids
Cores per node	112
MPI ranks per simulation	8
Wall-time per run	up to 24 h
Software stack	LifeX 2.1 / deal.II 9.7.1 / Trilinos / MPICH 4.0 / HDF5 1.14.5
Container runtime	Apptainer 1.4.5 (image `lifex_env.sif`)

Table 2: Computational resources for the simulation campaign on Leonardo.

2.2.2. Containerization and build process

The compute nodes of Leonardo do not expose a pre-configured HPC software stack through the `module` command, which only loads base system libraries. A native build of `LifeX` on the cluster would therefore require manually compiling `deal.II` 9.7 together with all its required backends[1], an operation that is both fragile and not portable across nodes. The entire toolchain was therefore packaged into an Apptainer image and executed through the container runtime available on the compute nodes (Apptainer 1.4.5), so that the build environment is identical to the execution environment.

[1] `Trilinos` (used by `LinearAlgebraTrilinos` and the `sacado_dfad` AD operators), PETSc, p4est, VTK, Boost, and HDF5

Identifying an image that simultaneously satisfies all `LifeX` dependencies required iterating over several candidates, summarized in Table 3, encountering a series of incompatibilities:

- the Spack stack pre-installed on Leonardo provides `dealii@9.6.2`, but only with the `~trilinos` variant, which is incompatible with `LifeX` 2.1;
- the official `dealii/dealii` Docker images expose `deal.II` 9.7 with all required backends, but lack the `boost_filesystemConfig.cmake` component required by the `LifeX` CMake configuration;
- an earlier MOX `mk` container shipping `deal.II` 9.5.1 fails to compile `LifeX` 2.1 due to API changes in `get_mpi_communicator`, `CellStatus`, and `AffineConstraints` that occurred upstream between 9.5 and 9.7

The image finally adopted, `mk-dealiimaster_latest.sif`, is the MOX MK container with the `deal.II`module tracking the upstream master branch (`deal.II` 9.7), together with matching `vtk`, `boost`, `trilinos`, `petsc`, `p4est`, and `hdf5` modules, activated inside the container by sourcing `/u/sw/etc/profile` and loading the relevant modules.

Stack	Components	Outcome
Spack `dealii@9.6.2` (Leonardo)	deal.II `~trilinos`, PETSc, MPICH	Missing Trilinos/Sacado required by LifeX 2.1
`dealii/dealii` Docker images (`v9.7.1-jammy`, `master-jammy`, `v9.7.1-noble`)	deal.II 9.7.1 with Trilinos, PETSc, p4est, VTK, HDF5	`boost_filesystemConfig.cmake` missing in the image
MOX `mk` container (deal.II 9.5.1)	deal.II 9.5.1, Boost 1.76, VTK, HDF5	API mismatch: LifeX 2.1 requires deal.II $\geq$ 9.7
`mk-dealiimaster_latest.sif`	MK toolchain with deal.II master ($\geq$ 9.7), Boost, VTK, Trilinos, PETSc, p4est, HDF5	**Adopted** for the simulation campaign

Table 3: Container strategies evaluated for building LifeX on Leonardo.

The CMake configuration is invoked through `apptainer exec -cleanenv -bind`, which strips host environment contamination and mounts the workspace inside the container. Because the simulation campaign is purely two-dimensional, the build is configured with `-DLIFEX_DIM=2` and restricted to the two example targets actually used, `example_fluid_dynamics_airfoil` and `example_fluid_dynamics_external_flow`. A global `make all` cannot be used in 2D because some files in the test suite (e.g. `tests/multidomain_laplace/laplace.cpp`) rely on overloads of `GridTools::rotate` that exist only for three-dimensional triangulations, and the resulting compilation error stops the whole build.
The complete build script is reported in Listing 3.

```
#!/bin/bash
#SBATCH -A EUHPC_D35_025
#SBATCH -p dcgp_usr_prod
#SBATCH --time=00:24:00
#SBATCH -N 1
#SBATCH --ntasks-per-node=112
#SBATCH --job-name=lifex_build

set -euo pipefail

WORK_DIR="$HOME/Ice_acceleration_model_on_airplane_wing"
SIF="${WORK_DIR}/containers/mk-dealiimaster_latest.sif"
SRC_SUBDIR="external/lifex-public"
BUILD_SUBDIR="external/lifex-public/build_2d"

rm -rf "${WORK_DIR}/${BUILD_SUBDIR}"

singularity exec --cleanenv \
  --bind "${WORK_DIR}:/work" \
  --bind /dev/shm:/dev/shm \
  "${SIF}" bash -c "
    source /u/sw/etc/profile
```

```
    module load gcc-glibc dealii vtk

    cmake -B /work/${BUILD_SUBDIR} -S /work/${SRC_SUBDIR} \
      -DCMAKE_BUILD_TYPE=Release \
      -DLIFEX_DIM=2 \
      -DDEAL_II_DIR=\${mkDealiiPrefix} \
      -DLin_ALG=Trilinos

    cmake --build /work/${BUILD_SUBDIR} \
      --target example_fluid_dynamics_airfoil \
               example_fluid_dynamics_external_flow \
      -j 112
  "
```

Listing 3: SLURM build script for LifeX on Leonardo (`job_build_leonardo.sh`). The Apptainer image is invoked through the legacy `singularity` command alias.

2.2.3. Mesh generation and pre-processing pipeline

The computational meshes are generated with Gmsh from parametric `.geo` scripts, driven by a set of Python generators (`generate_geo*.py`, `naca_generator_v*.py`) that expose the airfoil profile (NACA 0006 through NACA 0018), the domain extent, and the boundary-layer structure as input parameters.

The resulting mesh is hybrid: a structured layer of quadrilaterals wraps the airfoil surface to resolve the boundary layer, while the outer region is filled with unstructured triangles. Boundary entities follow a fixed tagging convention (`1` = inlet, `2` = outlet, `3` = airfoil wall) matching the tags expected by the `LifeX` configuration (Section 2.1), and the mesh is exported in the MSH 2.2 format read natively by `deal.II`.

In accordance with the wall-resolved turbulence modeling approach described in Section 2.1.3, the near-wall region is carefully refined. The height of the first cell layer is adjusted until the estimated dimensionless wall distance satisfies the criterion $y^+ \lesssim 1$ at a Reynolds number of $\mathrm{Re} = 10^6$. This ensures that the use of a homogeneous Dirichlet condition for $\tilde{\nu}$ is physically valid.

Because a single malformed mesh wastes hours of allocated CPU time, every mesh is screened by a pre-processing pipeline before submission. Four independent Python checks are run in sequence:

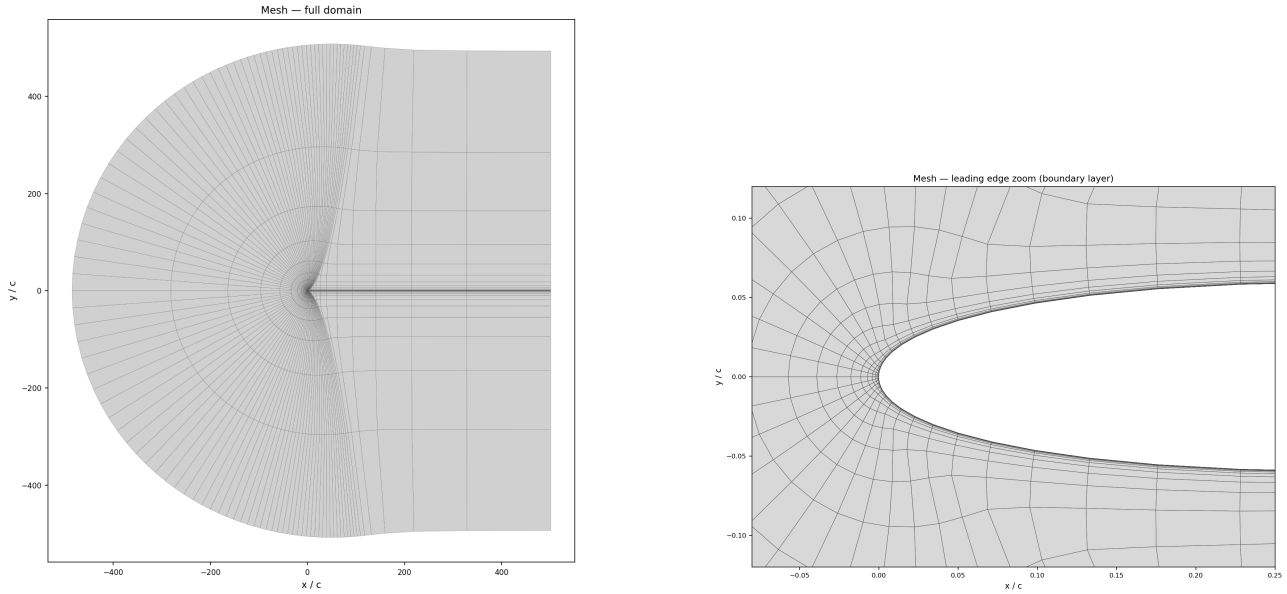
- `mesh_info.py` reports the node and element counts grouped by element type and physical tag and verifies that the inlet, outlet, and airfoil tags are all present;
- `check_extents.py` computes the bounding box of each tagged boundary, confirming that the inlet, outlet and airfoil entities are positioned where the geometry expects them;
- `check_jacobians.py` reads the mesh through `meshio` and computes the signed area of every quadrilateral via the cross product of its diagonals, flagging inverted or non-convex cells that would produce a negative Jacobian;
- `check_yplus.py` estimates the wall-normal y^+ at the first node by measuring the height of the cells adjacent to the airfoil edges and applying a flat-plate skin-friction correlation ($u_\tau/U_\infty \approx 0.036$), consistent with the estimate in Section 2.1.3.

Any mesh that fails one or more of these checks is automatically discarded and regenerated. An example of a mesh that has passed this quality control pipeline, used for the NACA 0012 simulations, is shown in Figure 2.

2.2.4. Simulation execution and post-processing

Each simulation is launched as a single-node SLURM job that runs the `LifeX` executable inside the Apptainer container with `mpirun -n 8`, distributing the mesh across eight MPI ranks on the `dcgp_usr_prod` partition.

The workspace is bind-mounted into the container and the run-time configuration is supplied as a JSON file, following the parameter structure described in Section 2.1: the turbulence model is activated through `"Turbulence`



(a) Overall domain

(b) Leading edge detail

Figure 2: Computational mesh for the NACA 0012 airfoil. (a) Overall domain, with a structured quadrilateral boundary layer around the profile and an unstructured triangular outer region. (b) Detail of the wall-resolved boundary layer at the leading edge, refined to give $y^+ \lesssim 1$ at $\mathrm{Re} = 10^6$.

`model": "Spalart-Allmaras"`, and the Reynolds number is set implicitly through the kinematic viscosity ($\nu = 10^{-6}$ with $U_\infty = 1$ and chord $c = 1$, giving $\mathrm{Re} = 10^6$). Before execution, the configuration file is copied into the run directory `output/run_${SLURM_JOB_ID}/` as `config_used.json`, so that every result remains paired with the exact parameters that produced it.

During the run, the solver generates two primary forms of output: a time history of the aerodynamic coefficients, which is appended to `force.csv`, and the full velocity, pressure, and turbulent-viscosity fields, saved as HDF5 files with corresponding XDMF descriptors for visualization in ParaView. The complete SLURM submission script orchestrating this process is reported in Listing 5.

```
#!/bin/bash
#SBATCH -A EUHPC_D35_025
#SBATCH -p dcgp_usr_prod
#SBATCH --time=24:00:00
#SBATCH -N 1
#SBATCH --ntasks-per-node=8
#SBATCH --job-name=lifex_run
#SBATCH -o output/run_%j/solver.log

set -euo pipefail

WORK_DIR="$HOME/Ice_acceleration_model_on_airplane_wing"
SIF="${WORK_DIR}/containers/mk-dealiimaster_latest.sif"
EXEC="/work/external/lifex-public/build_2d/examples/fluid_dynamics_airfoil
      /lifex_example_fluid_dynamics_airfoil"
CONFIG="/work/cases/naca0012/config_new_method.json"
RUN_DIR="${WORK_DIR}/output/run_${SLURM_JOB_ID}"

mkdir -p "${RUN_DIR}"
cp "${WORK_DIR}/cases/naca0012/config_new_method.json" "${RUN_DIR}/
    config_used.json"

singularity exec --cleanenv \
  --bind "${WORK_DIR}:/work" \
```

```
  --bind /dev/shm:/dev/shm \
  "${SIF}" bash -c "
    source /u/sw/etc/profile
    module load gcc-glibc dealii vtk
    export LD_LIBRARY_PATH=/work/external/lifex-public/build_2d/lib:\
       $LD_LIBRARY_PATH

    cd /work/output/run_${SLURM_JOB_ID}
    mpirun -n 8 ${EXEC} ${CONFIG}
  "
```

Listing 4: SLURM run script for a single simulation (`job_run_leonardo.sh`).

The field output is inspected in ParaView, while the coefficient time histories are plotted with the `plot_forces.py` script (matplotlib with the `Agg` backend, so that figures are rendered without a display on the compute nodes).

Figures 3 and 4b show representative output for the NACA 0012 at $\alpha = 4°$ and $\mathrm{Re} = 10^6$. The pressure coefficient field (Figure 3) exhibits the suction peak on the upper surface near the leading edge that generates the net lift; the velocity field (Figure 4a) shows the flow accelerating over the suction side and the thin viscous wake downstream of the trailing edge; the spanwise vorticity (Figure 4b) isolates the shed boundary layers in the wake.

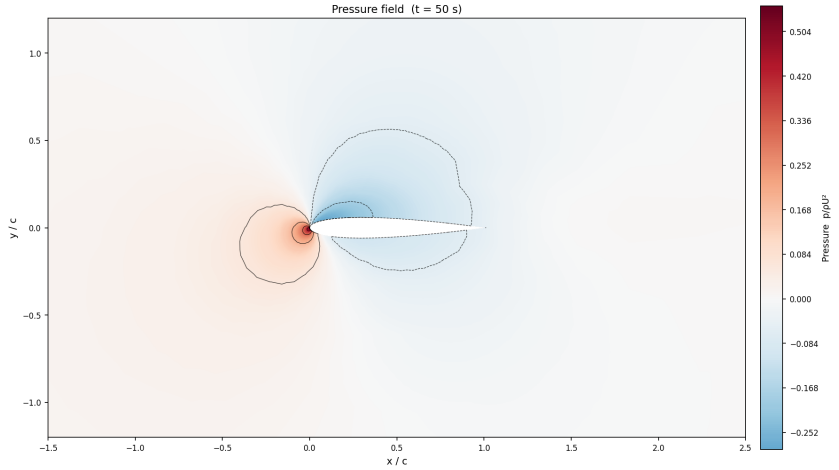
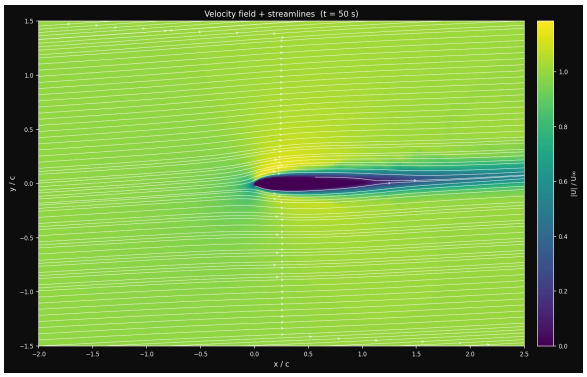
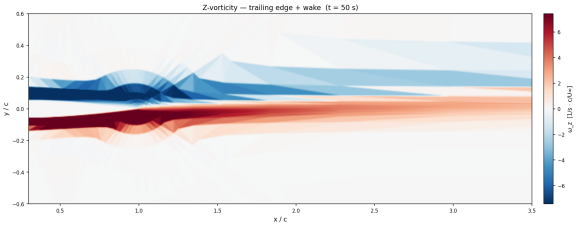


Figure 3: Pressure coefficient field around the NACA 0012 at $\alpha = 4°$, $\mathrm{Re} = 10^6$. The suction peak on the upper surface near the leading edge and the stagnation region on the lower side generate the net lift.



(a) Velocity magnitude with superimposed streamlines.



(b) Spanwise vorticity in the wake.

Figure 4: Flow field around the NACA 0012 at $\alpha = 4°$, $\mathrm{Re} = 10^6$. (a) The flow accelerates over the suction side and produces a thin viscous wake downstream of the trailing edge. (b) The spanwise vorticity isolates the shed boundary layers.

2.2.5. Dataset generation for the surrogate model

The simulation is not an end in itself: its role is to build the training set for the convolutional surrogate model presented in Section 3.

To build a comprehensive dataset, the parameter space was systematically explored. For each airfoil profile in a family of symmetric NACA sections (NACA 0006 to NACA 0018), the angle of attack was swept over the range $\alpha \in \{-14°, -12°, \ldots, +14°\}$ using templated configuration files (`config_isa_aoa_*.json`).

For each configuration, the pipeline produces a single label: the converged lift coefficient, C_L. These labels populate the `Y_cl` array of the final dataset, which is stored in the compressed NumPy format (`.npz`). The corresponding inputs are the Signed Distance Function representation of each geometry and the scalar pair of physical parameters (Re, α).

Computing the lift labels with the turbulence-resolved solver, rather than with a low-fidelity panel method, is what allows the surrogate to be trained against high-fidelity data in the high-incidence regime where thin-airfoil theory no longer holds.

The same workflow extends to the iced-airfoil geometries. A modified leading edge produced by an icing model could be created in future workflows and processed through the identical execution and post-processing chain, so that the dataset can later be augmented with iced profiles without any change to the simulation pipeline. The complete workflow is summarized in Figure 5.

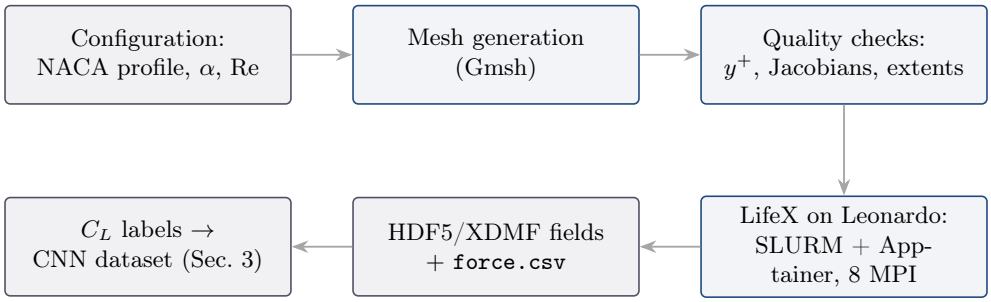


Figure 5: End-to-end simulation pipeline (top row left to right, then bottom row right to left). Mesh generation and quality control run interactively on the login node; the solver runs as a SLURM batch job inside the Apptainer container on the compute nodes; the resulting lift coefficients feed the convolutional surrogate model of Section 3.

3. Convolutional Neural Network

Building upon the dataset established in the preceding Section 2, this Section introduces a surrogate model designed to approximate the results of CFD simulations with a significant reduction in computational time.

Our approach is centered on a **Convolutional Neural Network (CNN)**, a deep learning architecture adept at learning the complex physical relationships that govern aerodynamic performance directly from simulation data.

Specifically, the network is designed to predict the lift coefficient (C_L) as a function of three key inputs:

1. the airfoil shape, represented as a **Signed Distance Function**;
2. the angle of attack (α);
3. the Reynolds number (Re).

By training on the generated simulation results, the CNN effectively learns to emulate the fluid dynamics solver, enabling near-instantaneous predictions.

The following sections will provide a comprehensive overview of this CNN model, from its underlying mathematical principles to its custom implementation designed for high-performance computing.

3.1. Mathematical Background

This section details the mathematical foundations of our surrogate modeling approach.
We first introduce the **Signed Distance Function (SDF)**, which transforms the airfoil's complex geometry into a structured grid suitable for a neural network. We then provide a comprehensive breakdown of the **Convolutional Neural Network architecture**, explaining the mathematical operations within each layer and the physics-informed training algorithm used to learn the mapping from geometry and flow conditions to the aerodynamic lift coefficient.

3.1.1. Geometric Representation with Signed Distance Function

To construct a deep learning model capable of predicting aerodynamic coefficients, we need a robust method for representing the geometry of the airfoil $\Omega \subset \mathbb{R}^2$ and its boundary $\partial\Omega$ in a format suitable for Convolutional Neural Networks. Traditional boundary representations, such as coordinates of boundary points, are challenging to process with standard convolutional operators because they lack fixed spatial grids. To address this limitation, we represent the airfoil geometry using a **Signed Distance Function**, obtaining for each airfoil the result shown in Figure 6.

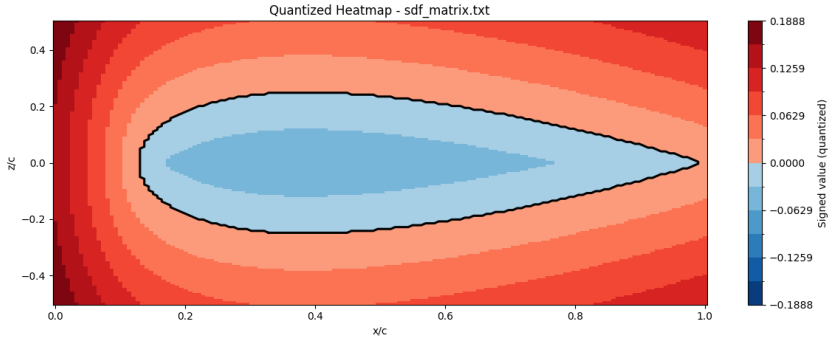


Figure 6: The Signed Distance Field representation of a NACA 0012 airfoil, discretized on a uniform grid. Negative values (blue) correspond to the airfoil interior, positive values (red) to the exterior while the boundary is represented by the zero-level contour (black line).

The SDF $\Phi : \mathbb{R}^2 \to \mathbb{R}$ maps any point $P = (x, z)$ in the spatial domain to its signed distance from the boundary $\partial\Omega$:

$$\Phi(P) = \begin{cases} -d(P, \partial\Omega) & \text{if } P \in \Omega \\ d(P, \partial\Omega) & \text{if } P \notin \Omega \end{cases} \tag{10}$$

where $d(P, \partial\Omega)$ represents the Euclidean distance from the point P to the boundary $\partial\Omega$:

$$d(P, \partial\Omega) = \inf_{Q \in \partial\Omega} \|P - Q\|_2 \tag{11}$$

Using this definition, the boundary of the airfoil is implicitly defined as the zero-level set of the function:

$$\partial\Omega = \{P \in \mathbb{R}^2 \mid \Phi(P) = 0\} \tag{12}$$

while the interior corresponds to negative values ($\Phi(P) < 0$) and the exterior to positive values ($\Phi(P) > 0$).

In our numerical implementation, the boundary $\partial\Omega$ is defined as a closed polygon composed of an ordered set of n vertices $\{V_0, V_1, \ldots, V_{n-1}\}$.
To guarantee scale invariance, the coordinates of the airfoil profile are first normalized by the chord length c, defined as the Euclidean distance between the leading edge point $(x_{\min}, z|_{x_{\min}})$ and the trailing edge point $(x_{\max}, z|_{x_{\max}})$:

$$c = \sqrt{(x_{\max} - x_{\min})^2 + (z_{\text{at } x_{\max}} - z_{\text{at } x_{\min}})^2} \tag{13}$$

where $x_{\max}$, $x_{\min}$, $z_{\max}$, and $z_{\min}$ are the extreme coordinates computed over the boundary vertices. Each vertex is then scaled by dividing its coordinates by c:

$$V_i = \left(\frac{x_{V_i}}{c}, \frac{z_{V_i}}{c}\right) \tag{14}$$

The boundary $\partial\Omega$ is then represented as the union of n linear segments $S_i = [V_i, V_{i+1}]$ (with $V_n = V_0$). The minimum distance from a grid point P to the boundary $\partial\Omega$ is computed by finding the minimum distance to each individual segment S_i:

$$d(P, \partial\Omega) = \min_{i=0,\dots,n-1} d(P, S_i) \tag{15}$$

For a given segment S_i defined by the endpoints $A = V_i$ and $B = V_{i+1}$, any point on the infinite line passing through A and B can be parameterized as $P(t) = A + t(B - A)$ for $t \in \mathbb{R}$. The orthogonal projection of the point P onto this line is obtained by minimizing the distance $\|P - P(t)\|^2$, which yields the projection parameter:

$$t^* = \frac{(P - A) \cdot (B - A)}{\|B - A\|_2^2} \tag{16}$$

To restrict the projection to the finite line segment S_i, t^* is clamped to the interval $[0, 1]$:

$$t = \max(0, \min(1, t^*)) \tag{17}$$

The projection point on the segment is then $P_{\text{proj}} = A + t(B - A)$, and the distance to the segment is:

$$d(P, S_i) = \|P - P_{\text{proj}}\|_2 \tag{18}$$

To determine the sign of the distance field, we implement the **Winding Number Algorithm** to check whether a grid point P lies inside or outside the closed boundary. The winding number $wn(P)$ computes the number of times the boundary curve wraps around the point P. For a closed polygon, it is computed by casting a horizontal ray starting from P and pointing in the positive x-direction, and counting the signed crossings of this ray with the polygon edges:

- an **upward crossing** occurs if an edge $S_i = [V_i, V_{i+1}]$ crosses the horizontal ray from below, i.e., $V_{i,z} \leq z_P < V_{i+1,z}$ and P lies to the left of the directed line segment;
- a **downward crossing** occurs if an edge S_i crosses the horizontal ray from above, i.e., $V_{i+1,z} \leq z_P < V_{i,z}$ and P lies to the right of the directed line segment.

The relative position of the point P with respect to the directed segment $[A, B]$ is determined mathematically using the 2D cross product:

$$\text{cross}(P, A, B) = (A_x - x_P)(B_z - z_P) - (B_x - x_P)(A_z - z_P) \tag{19}$$

If $\text{cross}(P, A, B) > 0$, the point P lies to the left of the directed segment, whereas if $\text{cross}(P, A, B) < 0$, it lies to the right. The winding number is incremented by 1 for every upward crossing and decremented by 1 for every downward crossing. A point P is strictly inside the domain Ω if and only if the total winding number is non-zero:

$$P \in \Omega \iff wn(P) \neq 0 \tag{20}$$

Once the signed distance is computed for any point, we project it onto a discrete uniform grid. The spatial domain is discretized into a Cartesian grid of size $N \times N = 150 \times 150$ over the bounding box $[X_{\min}, X_{\max}] \times [Z_{\min}, Z_{\max}]$, with:

$$X_{\min} = -0.15, \quad X_{\max} = 1.01, \quad Z_{\min} = -0.12, \quad Z_{\max} = 0.12 \tag{21}$$

The grid spacing is defined by:

$$\Delta x = \frac{X_{\max} - X_{\min}}{N - 1}, \quad \Delta z = \frac{Z_{\max} - Z_{\min}}{N - 1} \tag{22}$$

For each grid node indexed by (row, col) for $row, col = 0, \dots, N - 1$, the corresponding spatial coordinates are given by:

$$x_{col} = X_{\min} + col \cdot \Delta x, \quad z_{row} = Z_{\max} - row \cdot \Delta z \tag{23}$$

This discretized representation yields a 2D matrix of shape 150×150, where each entry contains the signed distance value. This matrix provides a spatially structured representation of the geometry that preserves boundary details, making it perfectly suited as an input to the convolutional branch of the neural network.

3.1.2. Convolutional Neural Networks for Aerodynamic Prediction

The architecture of our **Convolutional Neural Network** is designed as a multi-input model to effectively process both the spatial geometric data and the scalar physical parameters. A high-level schematic of this design, inspired by the model presented by Cuozzo et al. (2026), is depicted in Figure 7.

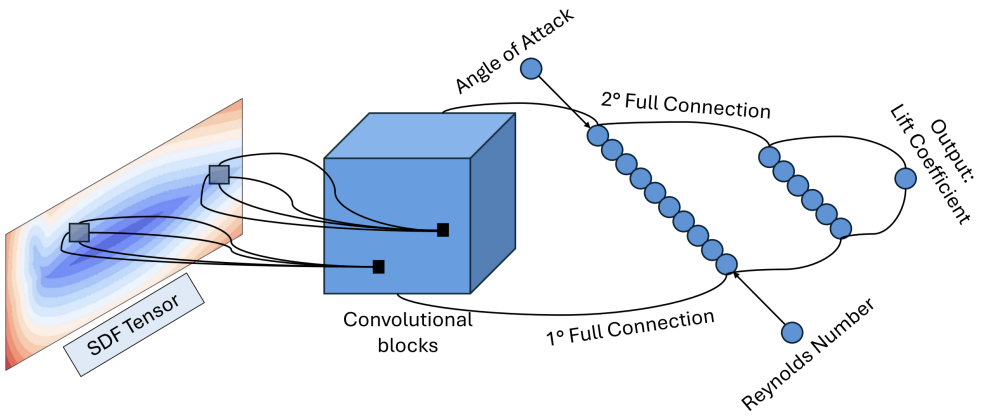


Figure 7: Architecture of the Convolutional Neural Network used for aerodynamic prediction.

The network is logically divided into three main components:

1. a convolutional feature extractor, responsible for interpreting the airfoil geometry. It accepts the 2D SDF matrix as input and processes it through a series of `Conv2DLayer`s, each followed by a `LeakyReLULayer` activation. This stack of layers hierarchically extracts relevant spatial features, such as leading-edge curvature and surface details. The resulting multi-channel feature maps are then transformed into a one-dimensional vector by a `FlattenLayer`, producing a learned representation of the airfoil's shape;

2. a feature fusion stage, in which, the flattened geometric feature vector from the convolutional branch is concatenated with the scalar physical parameters (the angle of attack and the Reynolds number) tensor using a `ConcatenateLayer`. This creates a single feature vector that contains comprehensive information about both the airfoil's shape and the current flow conditions;

3. finally the unified vector is passed to the regression head, which consists of a sequence of `DenseLayer`s. This fully-connected network maps the combined high-level features to a single scalar output, which represents the predicted lift coefficient, C_L.

Convolutional Layer

The **Convolutional Layer** (`Conv2DLayer`) is an operator specifically designed to process grid-like data such as the 2D SDF. Its primary function is to detect local spatial patterns within the input by applying a set of learnable filters. The difference with the dense layer, which treats all input pixels independently, is that a convolutional layer preserves the spatial relationship between pixels by operating on localized neighborhoods.

At its core, the operation is a discrete 2D convolution, which can be understood as a *sliding window dot product*. A small matrix of learnable weights, known as the kernel (K), slides over the input feature map (I). At each position, an element-wise multiplication between the kernel and the overlapping region of the input is performed, and the results are summed up to produce a single pixel in the output feature map (O).

For a single-channel input I of size $H \times W$ and a kernel of size $F \times F$, the value of the output O at position (i, j) is given by:

$$O(i,j) = \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} I(i-m, j-n) \cdot K(m,n) + b \tag{24}$$

where b is a learnable scalar bias term applied to the entire output feature map.

The key principle here is parameter sharing: the *same* kernel K and bias b are used across all positions of the input, allowing the network to detect the same feature regardless of where it appears in the SDF[2].

In deep networks, layers typically process multi-channel inputs. If the input feature map has C_{in} channels[3], the kernel must be three-dimensional, with a size of $F \times F \times C_{in}$. The convolution is then performed across all channels simultaneously. The results from each channel are then summed together to produce a single value for the 2D output map.

To generate an output with C_{out} feature maps, we simply apply C_{out} independent kernels. Each kernel is responsible for learning a different spatial feature, allowing the layer to build a complete representation of the input geometry.

The behavior of the convolution is controlled by two hyperparameters:

- the stride (S) defines the step size the kernel matrix as it traverses the input grid. While a unit stride ($S = 1$) evaluates adjacent localized neighborhoods sequentially, a stride of $S > 1$ skips spatial coordinates, downsampling the spatial resolution of the resulting feature map;
- padding (P)[4] consists in appending a perimeter of zero-valued elements around the input grid boundaries. This can be used to control the output dimensions and mitigate the loss of information at the edges.

The dimensions of the output feature map ($H_{out} \times W_{out}$) are determined by the input dimensions ($H_{in} \times W_{in}$), the kernel size (F), the stride (S) and the padding (P) using the following formula:

$$W_{out} = \frac{W_{in} - F + 2P}{S} + 1 \quad \text{and} \quad H_{out} = \frac{H_{in} - F + 2P}{S} + 1 \tag{25}$$

From an implementation perspective, this operation is typically realized as a set of nested loops that iterate over each position in the output map, each element of the kernel, and each input channel, performing the required multiply-accumulate operations[5].

Non-Linear Activation (Leaky ReLU)

After each `Conv2DLayer` or `DenseLayer` performs its linear operation, an activation function applies a non-linear transformation element-wise to its output. Without this, a deep neural network, regardless of its number of layers, would behave as a single, large linear model. The introduction of non-linearity is what gives the network its power to approximate arbitrarily complex, non-linear relationships, such as the one between airfoil geometry and lift coefficient.

In this project, a **Leaky Rectified Linear Unit** (Leaky ReLU) is implemented as the `LeakyReLULayer`. This function, shown in Figure 8, is a popular and effective choice in modern deep learning due to its computational efficiency and its ability to mitigate some of the common problems encountered during training.

The Leaky ReLU function is defined mathematically as:

$$\text{LeakyReLU(x)} = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \tag{26}$$

where α is a small, positive constant (a hyperparameter, typically set to a value like 0.01).

The function behaves as a simple identity for all positive input values. For negative input values, instead of mapping them to zero like the standard ReLU function, it allows a small, non-zero gradient to pass through.

The primary advantage of the Leaky ReLU over the standard ReLU is its handling of negative inputs. In a standard ReLU, if a neuron's input is consistently negative, its gradient will always be zero. This means the

[2]In more details, if the kernel learns to recognize a leading edge curve of a profile, it will successfully detect the curve whether it appears at the top of the grid or in a bottom corner.

[3]This is not our case, since in the SDF we have only one input channel ($C_{in} = 1$).

[4]In our implementation, padding is set to 0.

[5]We will explain in a better way highlighting our C++ implementation in Section 3.2.3 regarding our implementation in C++

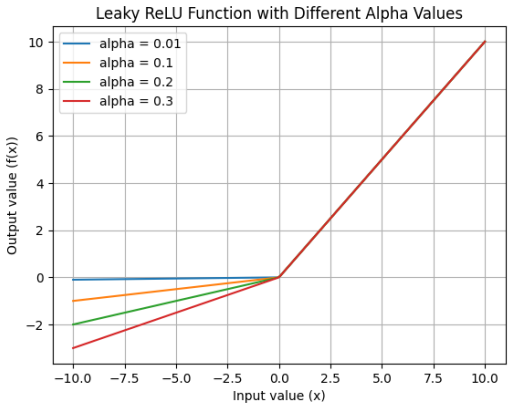


Figure 8: Example of Leaky ReLU function with different α.

neuron's weights will never be updated, and it can effectively "die", ceasing to contribute to the network's learning process. The Leaky ReLU prevents this "dying ReLU" problem by ensuring that a small gradient always exists, even for negative inputs.

From the backpropagation algorithm, the gradient of the Leaky ReLU is simple and computationally inexpensive to calculate:

$$\frac{d}{dx}\text{LeakyReLU(x)} = \begin{cases} 1 & \text{if } x > 0 \\ \alpha & \text{if } x \leq 0 \end{cases} \tag{27}$$

During the backward pass of training, this straightforward derivative is used to scale the incoming gradient from the subsequent layer:

- if the input to the activation function was positive during the forward pass, the gradient passes through unchanged;
- if it was negative, the gradient is scaled down by the factor α.

This simple conditional check makes the backward pass for the `LeakyReLULayer` extremely fast, contributing to the overall efficiency of the training process.

Structural Layers

After the convolutional network has extracted a hierarchy of spatial features, the resulting multi-channel feature maps must be prepared for integration with the scalar parameters and for final processing by the regression head. This preparation is handled by two complementary **structural layers**, the `FlattenLayer` and the `ConcatenateLayer`, that are parameter-free: they do not contain any learnable weights or biases since their sole purpose is to rearrange data tensors.

Flatten Layer

The `FlattenLayer` links the convolutional layers and the fully-connected layers. Convolutional layers output a 4D tensor with a shape of `[Batch Size, Channels, Height, Width]`, preserving spatial information. However, dense layers require their input to be a 2D tensor of shape `[Batch Size, Features]`, where each row is a one-dimensional vector representing a single sample.

The flatten operation reshapes an input tensor T_{in} with dimensions $[B \times C_{out} \times H_{out} \times W_{out}]$ into an output tensor T_{out} with dimensions $[B \times D]$, where the new feature dimension D is the product of the input's channel, height and width dimensions:

$$D = C_{out} \times H_{out} \times W_{out} \tag{28}$$

During the forward pass, this is a straightforward memory rearrangement.
During the backward pass for backpropagation, the `FlattenLayer` performs the inverse operation: it receives an incoming gradient tensor of shape $[B \times D]$ and reshapes it back to the original 4D shape $[B \times C_{out} \times H_{out} \times W_{out}]$. This ensures that the gradients are correctly propagated back to the preceding convolutional layer in the required spatial format.

Concatenate Layer

Once the geometric features have been flattened into a vector, they must be combined with the scalar physical parameters by the `ConcatenateLayer`, that takes multiple input tensors and joins them together along a specified axis[6].

Mathematically, given the flattened geometric feature tensor T_{flat} of shape $[B \times D_1]$ and the scalar input tensor T_{scalar} (containing angle of attack and Reynolds number) of shape $[B \times D_2]$, the Concatenate Layer produces an output tensor T_{concat} of shape $[B \times (D_1 + D_2)]$. This operation creates a unified feature vector that encodes both geometric and physical information for the network to make a prediction.

During backpropagation, the process is reversed. The incoming gradient tensor is split according to the original input sizes:
- the first D_1 elements of the gradient vector are passed back to the `FlattenLayer`'s branch;
- the D_2 elements corresponding to the scalar inputs are also computed, although they terminate at the input layer.

After this fusion step, the resulting unified tensor is ready to be processed by the final regression head of the network.

Dense Layer and final regression

The final computational stage of the network is handled by the **Dense Layer** (`DenseLayer`), known as a fully-connected layer. Its role is to learn the final mapping from the feature space to the target scalar value (the lift coefficient C_L).

A dense layer applies a linear transformation to its input vector. For an input vector $\mathbf{x} \in \mathbb{R}^{D_{in}}$, the layer computes an output vector $\mathbf{y} \in \mathbb{R}^{D_{out}}$ using a learnable weight matrix $W \in \mathbb{R}^{D_{out} \times D_{in}}$ and a learnable bias vector $\mathbf{b} \in \mathbb{R}^{D_{out}}$. The operation is defined as:

$$\mathbf{y} = W\mathbf{x} + \mathbf{b} \tag{29}$$

This operation performs a weighted sum of all input features to compute each output feature. Every neuron in the input layer is connected to every neuron in the output layer, allowing the network to model complex interactions between all elements of the input feature vector.

This layer is used in the regression head, which consists of a sequence of `DenseLayer`s, each followed by a `LeakyReLULayer` activation:
- the initial dense layers take concatenated feature vector as input. Their purpose is to find and amplify the non-linear correlations between the learned geometric features from the SDF and the raw physical parameters (α and Re);
- the last layer is configured to have a single output neuron ($D_{out} = 1$) and does not have a subsequent activation function. This single neuron's output directly corresponds to the network's prediction for the lift coefficient (C_L).

During the backward pass, the dense layer must compute three distinct gradients based on the incoming gradient from the subsequent layer $\frac{\partial L}{\partial \mathbf{y}}$, where L is the loss:
- the gradient with respect to the weights (W) is calculated as the outer product of the incoming gradient and the layer's input vector from the forward pass:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \mathbf{y}} \cdot \underbrace{\mathbf{x}^{\mathbf{T}}}_{\frac{\partial \mathbf{y}}{\partial \mathbf{W}}} \tag{30}$$

- the gradient with respect to the biases (b) is equal to the incoming gradient:

$$\frac{\partial L}{\partial \mathbf{b}} = \frac{\partial L}{\partial \mathbf{y}} \tag{31}$$

[6] In our case, this is the feature axis (axis 1)

- the gradient with respect to the input (x) must be passed back to the preceding layer. It is computed by multiplying the transposed weight matrix by the incoming gradient:

$$\frac{\partial L}{\partial \mathbf{x}} = W^T \cdot \frac{\partial L}{\partial \mathbf{y}} \tag{32}$$

These calculations are the fundamental operations that enable the network's regression head to learn from the training data.

Network Training

The process of training the neural network involves iteratively adjusting its parameters to minimize a loss function, which quantifies the discrepancy between the model's predictions and the desired outcomes. Our approach embeds physical knowledge directly into the training process, making our model a type of **Physics-Informed Neural Network (PINN)**. This is achieved through a composite loss function that balances empirical data accuracy with a theoretical physical constraint.

The total loss, L, is a weighted sum of data-driven term and a physics-informed term:

$$L = L_{\text{data}} + \lambda L_{\text{physics}} \tag{33}$$

where λ is a hyperparameter that controls the strength of the physical regularization.

The first component is the standard **Mean Squared Error** (MSE), which measures the difference between the network's predicted lift coefficients (P_i) and the ground-truth values from the CFD simulations (T_i):

$$L_{\text{data}} = \frac{1}{N} \sum_{i=1}^{N} (P_i - T_i)^2 \tag{34}$$

This term ensures the network learns to faithfully replicate the high-fidelity simulation data generated for this work.

The second component acts as a physical constraint, guiding the network's predictions to be consistent with established aerodynamic theory. This term is based on **Thin-Airfoil Theory**, which provides an analytical approximation for the lift coefficient at small angles of attack. The general form of this relationship is $C_L = 2\pi(\alpha - \alpha_{L=0})$, where $\alpha_{L=0}$ is the zero-lift angle of attack.

For symmetric airfoils, such as those in our dataset, the zero-lift angle of attack is, by definition, $\alpha_{L=0} = 0$. This simplifies the theoretical relationship to the linear form:

$$C_L \approx 2\pi\alpha \tag{35}$$

The physical loss in the network leverages this simplified form, penalizing deviations from this theoretical relationship:

$$L_{\text{physics}} = \frac{1}{N} \sum_{i=1}^{N} M_i \cdot (P_i - 2\pi\alpha_i)^2 \tag{36}$$

where M_i is a mask function, introduced because Thin-Airfoil Theory is only valid in the linear flight regime. Therefore, the mask is defined as:

$$M_i = \begin{cases} 1 & \text{if } |\alpha_i| \leq 10^\circ \\ 0 & \text{if } |\alpha_i| > 10^\circ \end{cases} \tag{37}$$

This ensures the physical constraint is only active at low angles of attack.
In the non-linear, high-angle-of-attack regime, the mask deactivates this term ($M_i = 0$), forcing the network to rely entirely on the empirical CFD data, which accurately captures the complex flow separation phenomena.

By explicitly building in the assumption of symmetry, this physics-informed regularizer provides a robust theoretical anchor for the network's predictions without being applied in regimes where its underlying assumptions would be violated.

Optimization with Adam

Once the backpropagation algorithm has computed the gradient of the loss function with respect to every trainable parameter in the network ($\nabla L(\theta)$, where θ represents all weights and biases), the final step in the learning process is to update these parameters. This is done using the **Adam (Adaptive Moment Estimation)** optimizer, which guides the parameters toward a minimum in the loss landscape.

Adam combines the advantages of the **RMSprop** and **Momentum** optimization methods. It maintains an exponentially decaying moving average of both the past gradients (first moment, the momentum term) and the past squared gradients (second moment, the adaptive learning rate term).

Let $g_t = \nabla L(\theta_{t-1})$ be the gradient at timestep t. The optimizer updates two moving averages:

- first moment (momentum):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{38}$$

- second moment (adaptive learning rate):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{39}$$

where β_1 and β_2 are hyperparameters (typically close to 1, e.g., 0.9 and 0.999 respectively).

These raw moment estimates are then bias-corrected to account for their initialization at zero:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \text{and} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{40}$$

Finally, the parameters are updated using these corrected estimates. The update rule for a parameter θ is:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \tag{41}$$

where η is the learning rate and ε is a small constant (e.g., 10^{-8}) to prevent division by zero.

The key intuition is that the update step is scaled by both a running average of the gradients (providing momentum to navigate flat regions) and an adaptive, per-parameter learning rate (scaling down updates for frequently changing parameters and scaling up updates for sparse ones). This adaptive nature makes Adam robust and generally leads to faster convergence compared to standard stochastic gradient descent.

3.2. Code Implementation

We implemented our CNN and SDF generator using an **object-oriented paradigm**. Figure 9 provides a comprehensive **UML class diagram** detailing the structural layout, member attributes and method signatures of the system.

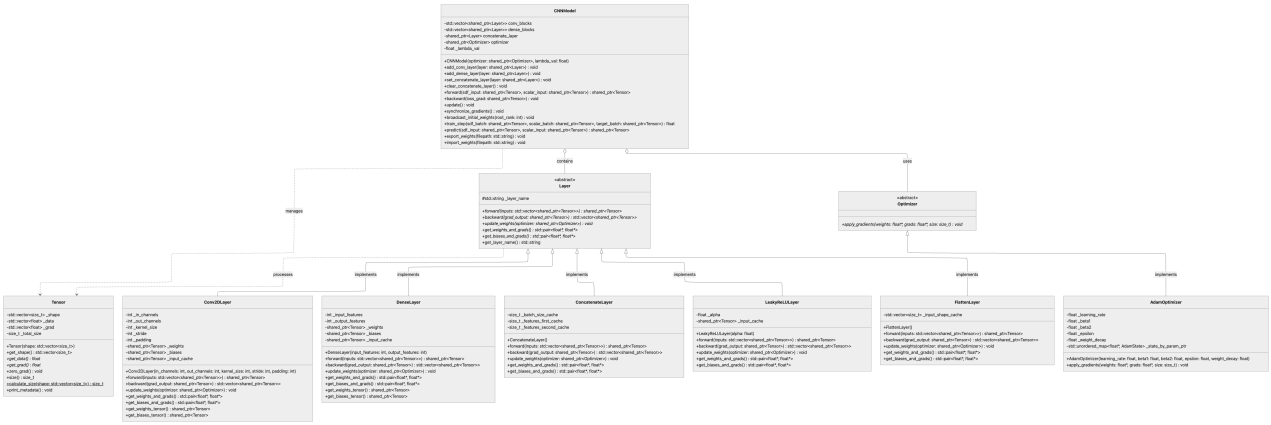


Figure 9: UML class diagram illustrating the object-oriented design of the C++ Convolutional Neural Network.

In the following sections, we decide to highlight the most interesting implementations of these components, detailing key choices and memory optimization layouts, as well as providing an explanation of the core functions implemented.

3.2.1. Geometric Preprocessing: SDF Generator

The generation of the Signed Distance Function is encapsulated within the `SDFGenerator` class, which maps airfoil geometries to structured 2D tensors suitable for convolutional processing.

To maximize CPU cache efficiency and minimize cache misses during spatial calculations, boundary vertices are stored in a `std::vector<Point>`, where `Point` is a simple structure holding `x` and `z` coordinates. This choice creates a single contiguous array for the boundary data.

Upon initialization, the constructor ensures scale invariance (as detailed in Section 3.1.1) by calling two private helper methods: `computeCord()` calculates the chord length c using `std::hypot` to prevent numerical overflow[7] and `normalizePoint()` scales all vertex coordinates accordingly[8].

With a normalized boundary in place, the `SDFGenerator` computes the signed distance for each point on the target grid. This is achieved through a set of optimized geometric kernel functions:

- `computeDistance(...) const` calculates the minimum Euclidean distance from a grid point to a boundary segment (as it is shown in Equations (16) and (17)). It includes a safety check for degenerate edges (where the two end-point of the segment coincide) to prevent division-by-zero errors;
- `getMinDistance(...) const` iterates through all boundary edges of the airfoil, calling for each one `computeDistance` and returning the absolute minimum distance found. Additionally, we used `static_cast<int>(airfoil.boundary.size())` to safely convert the unsigned vector size to a signed integer, preventing signed/unsigned comparison warnings;
- finally, the `windingAlgorithm(...) const` implements the Winding Number Algorithm previously explained (Section 3.1.1) to verify whether a grid point p lies inside or outside the closed boundary. This is computed by analyzing edge crossings and using the 2D cross product (Equation (19))[9], returning `true` if the point is inside the domain.

Once these values are computed for every grid point, the `mappingToMatrix` function converts the resulting 1D flattened SDF data into the required 150×150 2D matrix format and writes it to a file, utilizing the **RAII (Resource Acquisition Is Initialization)** paradigm via `std::ofstream outfile(filename)` that automatically closes the file descriptor when `outfile` goes out of scope, in order to prevent system resource leaks in case of exceptions.

The most computationally intensive part of the process is executing these kernels for each one of the $150 \times 150 = 22,500$ points on the grid. This task is a classic example of an *embarrassingly* parallel problem and it is implemented in the `generateTensorMPI` method, the details of which are provided in Section 3.3.

3.2.2. Core abstractions and memory management

To achieve high computational throughput while preserving a flexible design, the system introduces two core abstractions: a unified multi-dimensional tensor representation and a polymorphic layer interface.

Tensor

The foundational data structure of our CNN implementation is the `Tensor` class, which manages both forward activation data (`_data`) and backward gradients (`_grad`) as multi-dimensional arrays of single-precision floating-point numbers.

[7] This function scales intermediate values before performing the sum of squares, avoiding the premature overflow or underflow that can occur when computing the norm directly as $\sqrt{\Delta x^2 + \Delta z^2}$

[8] This normalization step will be valuable for future implementations aimed at studying the effect of ice accumulation on the lift coefficient.

[9] Implemented through the `crossProduct(Point p, Point a, Point b) const` with each point passed by value, in order improve performance for small `structs` by allowing them to be passed directly in CPU registers, avoiding memory indirection.

First of all, the constructor `Tensor(std::vector<size_t> shape)` initializes the shape descriptor and sets the `_total_size` member through a member **initializer list**. It pre-allocates and zero-initializes the storage vectors using `std::vector::resize`, that guarantees contiguous memory allocation. Direct access to this underlying memory is provided via the `get_data()` and `get_grad()` methods, which return raw pointers (`float*`) using the `.data()` member function.

Beyond raw pointer access, the class provides safe, native array-like indexing for flattened 1D elements through overloaded `operator[]` methods, offering both mutable and read-only (`const`-qualified) references. Utility functions are also optimized: gradient values are efficiently reset before each backward pass using the `std::fill` algorithm within the `zero_grad()` method.

Furthermore, the total element count is computed by `calculate_size()`, which is declared as a `static` method[10] and accepts a constant reference to avoid the overhead of copying the shape vector.

Finally, the class strictly adheres to `const` correctness, ensuring that utility methods like `print_metadata()` are executed safely without modifying the object's internal state.

Layer

The abstract base class `Layer`, defined in `Layer.hpp`, establishes a unified interface for all CNN components by leveraging runtime polymorphism to decouple the high-level network topology from the specific implementations of individual layers.

Runtime polymorphism is achieved through virtual functions and dynamic dispatch. When a class declares virtual methods, the compiler generates a virtual method table (`vtable`) containing pointers to these functions. Each object instance is implicitly equipped with a pointer (`vptr`) to its class's `vtable`. During execution, when a virtual method is called via a base class pointer, the program resolves the call dynamically: it dereferences the object's `vptr` to access the `vtable` and executes the overridden method corresponding to the actual derived type.

This mechanism is fundamental to our design. Rather than relying on conditional logic to handle the different mathematical operations, polymorphism allows the model to store and manage all layers uniformly within a single container (`std::vector<std::shared_ptr<Layer>`). In addition, this design ensures high scalability and the effortless integration of new layer types simply by inheriting from the `Layer` base class.

Following the idea of implementing the polymorphism, we declared a virtual destructor `~Layer() = default`, ensuring that when a derived layer object is destroyed via a base class pointer, the derived class's destructor is executed, preventing resource leaks of dynamically allocated memory.

The core operations of the network are declared as pure virtual functions (indicated by the `= 0` suffix), which must be overridden by every concrete layer subclass. Specifically, we implemented:

- `forward`, that computes forward activations, and `backward`, that performs backpropagation, both accepting and returning `std::shared_ptr` in order to guarantee automatic memory deallocation when the tensors are no longer needed;
- `get_weights_and_grads()` and `get_biases_and_grads()` that returns `std::pair<float*, float*>` packaging the weight/bias arrays and their corresponding gradients into a single return value. Returning non-const pointers allows the optimizer to modify the parameter values directly in place.

Finally, architectural decoupling is achieved through a forward declaration of the `Optimizer` class; this resolves potential circular dependencies while allowing the `update_weights(std::shared_ptr<Optimizer>)` method to delegate parameter updates to external optimization algorithms seamlessly.

3.2.3. Computational and activation layers

The computational layers manage the network's mathematical transformations and contain all trainable parameters, while activation layers introduce the necessary non-linear activation boundaries.

[10] This allows the method to be invoked without a class instance, for example, within a member initializer list.

DenseLayer

The concrete `DenseLayer` class implements the fully-connected linear transformation $\mathbf{y} = W\mathbf{x} + \mathbf{b}$, where W is the weight matrix and $\mathbf{b}$ is the bias vector, as described in details in the Section 3.1.2.

First of all, the constructor `DenseLayer(int input_features, int output_features)` allocates the weight and bias tensors utilizing `std::make_shared` rather than direct initialization via `std::shared_ptr(new Tensor(...))`. While this last one forces two separate heap allocations[11], `std::make_shared` performs a **single contiguous memory allocation** for both the reference-counting control block and the managed `Tensor` object, in order to reduce heap fragmentation and improve CPU cache locality by keeping the control metadata and the tensor data adjacent in memory.

To ensure gradient stability at the start of training, the weights are initialized according to the **Xavier/Glorot scheme**, drawing from a uniform distribution bounded by $\pm\sqrt{6/(N_{\text{in}} + N_{\text{out}})}$ using `std::default_random_engine` and `std::uniform_real_distribution<float>`.

During the `forward` pass, the input tensor is explicitly cached (`_input_cache`) to satisfy the mathematical requirements of backpropagation, as the input vector $\mathbf{x}$ must be retained to compute the weight updates as $\frac{\partial L}{\partial W} = \mathbf{x} \otimes \frac{\partial L}{\partial \mathbf{y}}$.

Moreover, the linear transformation $Y = XW^T + \mathbf{b}$ is evaluated through three nested loops:
- the outermost loop iterates over the batch size b;
- the middle loop iterates over the output features j;
- the innermost loop iterates over the input features i to evaluate the dot product $Y_{b,j} = \sum_i X_{b,i}W_{j,i} + b_j$.

The weight matrix W is stored in a transposed format of shape $(N_{\text{out}}, N_{\text{in}})$. This ensures that as the innermost loop increments its index over the input features, it accesses both the input tensor X and the weight tensor W with a unit stride. This design maximizes spatial locality, reduces cache misses, and enables the compiler to generate efficient SIMD vector instructions.

To conclude, also the `backward` method computes the gradients through three distinct sets of nested loops:
1. the gradient with respect to the inputs is calculated as $dX_{b,i} = \sum_j dY_{b,j}W_{j,i}$. In this loop structure, the weight matrix is traversed column-wise, resulting in non-contiguous memory access;
2. the gradient with respect to the weights is accumulated as $dW_{j,i} = \sum_b dY_{b,j}X_{b,i}$ using loops over j, i, and the batch dimension b. The innermost loop iterates over b, calculating the outer product of the output gradients and cached inputs;
3. the gradient with respect to the biases is accumulated as $dB_j = \sum_b dY_{b,j}$ by summing the output gradients across all batch elements for each output feature.

LeakyReLULayer

The concrete `LeakyReLULayer` class implements homonymous element-wise activation function shown in Equation (26). The negative slope parameter α is configured into the constructor `LeakyReLULayer(float alpha = 0.05f)`.

The interesting thing in this code is the use of the ternary operator:
- in the `forward` method, the element-wise activation is evaluated inside a contiguous loop using raw pointers `out_ptr[i] = (in_ptr[i] > 0.0f) ? in_ptr[i] : _alpha * in_ptr[i]`;
- in the `backward` method, that is used to evaluate the gradient with respect to the input by scaling the output gradient as shown in Equation (27), implemented as `din_ptr[i] = dout_ptr[i] * ((cache_ptr[i] > 0.0f) ? 1.0f : _alpha)`.

Because the derivative of the activation function is conditioned on the sign of the original input activations, caching the forward input tensor inside `_input_cache` is necessary to retrieve the values during the backward pass.

To conclude this class, because the activation layer contains no learnable parameters, the overridden method `update_weights` is implemented as a no-op. Moreover, the parameter accessors `get_weights_and_grads()` and `get_biases_and_grads()` return a pair of type-safe null pointers, `{nullptr, nullptr}`.

[11] One for the object and one for its control block.

Conv2DLayer

The concrete `Conv2DLayer` class extends the abstract `Layer` interface to perform 2D spatial convolutions on four-dimensional tensors, which logically represent **batch samples, channels, height and width**.

Upon instantiation, the constructor initializes the convolutional kernels using a **Gaussian Xavier scheme** to ensure variance stability across deep network layers. The standard deviation for this distribution is computed as $\sigma = \sqrt{2/(F_{\text{in}} + F_{\text{out}})}$, where the fan-in and fan-out dimensions are defined as $F_{\text{in}} = C_{\text{in}} \times K^2$ and $F_{\text{out}} = C_{\text{out}} \times K^2$, with K denoting the kernel size.

During the `forward` pass, the spatial dimensions of the output tensor are explicitly calculated using Equation (25) and the resulting tensor is allocated contiguously via `std::make_shared`. Because the system flattens all multi-dimensional tensors into 1D contiguous vectors for optimal memory layout, the layer utilizes a private inline helper function, `idx4`, to resolve 4D indices to 1D offsets. This enforces a strict row-major mapping:

$$\text{Offset}(a, b, c, d) = [(a \times \text{dim}_b + b) \times \text{dim}_c + c] \times \text{dim}_d + d \tag{42}$$

where a, b, c, d represent the target indices across the batch, channel, and spatial dimensions.

The forward convolution operation is executed through seven-nested loop. The outer four loops iterate over the batch elements n, output channels oc and spatial output coordinates oh and ow, while the inner three loops traverse the input channels ic and kernel dimensions kh and kw. To handle boundaries without incurring the overhead of allocating memory for physical zero-padding, the implementation uses implicit padding. Source coordinates are dynamically evaluated at each step as $ih = oh \times S + kh - P$ and $iw = ow \times S + kw - P$:

- if these fall outside the input boundaries, the multiplication is safely bypassed using a `continue` statement;
- otherwise, the flattened indices are retrieved via `idx4`, and the kernel-input products are accumulated into a running sum initialized with the appropriate channel bias.

Finally, the `backward` method evaluates gradients using two distinct loop structures:

- the first structure computes the parameter gradients: the bias gradient (dB) aggregates the output gradients directly, while the weight gradient (dW) accumulates the product of the output gradients and the cached forward inputs;
- the second structure computes the gradient with respect to the input (dX) by performing a full convolution between the output gradients and the flipped kernel filters. To achieve this, the loop accesses the weights using inverted spatial coordinates:

$$kh_{\text{orig}} = K - 1 - kh, \quad kw_{\text{orig}} = K - 1 - kw \tag{43}$$

3.2.4. Structural Layers

The **structural layers** are parameter-free components designed to rearrange and route data tensors across the branches of the network.

FlattenLayer

The `FlattenLayer` class extends the base `Layer` interface to perform a logical reshaping of the 4D feature maps (generated by the convolutional branch) into a 2D tensor suitable for the subsequent `ConcatenateLayer`.

The `forward` method calculates the flattened size $F = C \times H \times W$ and allocates a 2D output tensor of shape $[B \times F]$ via `std::make_shared`. Since the flatten operation is basically a reshape, the data is copied from the input to the output tensor sequentially in a single loop, preserving the element order across contiguous memory arrays.

The `backward` method receives the 2D gradient tensor `grad_output` (of shape $[B \times F]$), validates the batch size consistency, allocates a 4D gradient tensor matching the cached shape `_input_shape_cache`[12] and performs the inverse sequential copy `grad_in_ptr[i] = grad_out_ptr[i]`, reconstructing the 4D gradient layout for the preceding convolutional layer.

[12] This member attribute of type `std::vector<size_t>` preserves the original spatial dimensions ($C \times H \times W$) to reconstruct and propagate the gradient back.

ConcatenateLayer

The `ConcatenateLayer` class is used to obtain a single tensor that combines the information from the **Convolutional Layer** and the scalar values of the Reynolds number and angle of attack.

Beyond the mathematical formalism about the shape of the matrices described in a detailed way in Section 3.1.2, the merge operations are implemented with nested loops:

- the outer loop iterates over the batch size n, computing the row offsets for the first input (`n * _features_first_cache`), the second input (`n * _features_second_cache`) and the output (`n * (_features_first_cache + _features_second_cache)`);
- two inner loops then copy the data, ensuring row-wise contiguous layout.

During the `backward` method, this operation is inverted. The layer receives a combined gradient tensor of shape $[B \times (D_1 + D_2)]$ and must route the gradients back to their respective origins, reversing the copy logic: for each batch element, it copies the first D_1 components of the output gradient to the first input gradient buffer and the remaining D_2 components to the second input gradient buffer. Both elements are then returned in a standard vector `{grad_first, grad_second}`.

3.2.5. Training pipeline, loss and optimizers

The network's optimization and training pipeline is coordinated by mathematical loss functions and state-tracking optimization solvers.

Loss

Unlike the object-oriented structure of the computational layers, the **physics-informed loss function** and its corresponding gradient are implemented as non-member utilities within the `Loss` namespace. This design choice avoids the overhead of object instantiation and maintains a functional style for stateless operations.

In the `forward` method[13], the composite physics-informed loss, described in Section 3.1.2, is computed in a single loop using raw pointers retrieved via `get_data()` to ensure contiguous memory traversal. When computing the physics-informed penalty, the masking logic from Equation (37) is applied: the linear regime limit is saved in an anonymous namespace at the beginning of the file, in order to restrict the scope of this constant. The average data and physics terms are computed and normalized by the number of elements using `static_cast<float>(n)`

Finally, the `backward` method is implemented in a similar way.

Optimizer and Adam Optimizer

Similar to the layer architecture, the optimization logic is strictly decoupled from the neural network topology via an abstract `Optimizer` base class. This unified interface ensures that different numerical solvers can be swapped seamlessly, exposing a single pure virtual function (`apply_gradients(float* weights, float* grads, size_t size)`) to handle dynamic dispatch interface that must be implemented by concrete subclasses.

The only concrete implementation of this interface is the `AdamOptimizer`, which executes the **Adaptive Moment Estimation** algorithm detailed in Section 3.1.2.

To map specific weight and bias arrays to their corresponding optimization states without coupling the layers to the optimizer, the class utilizes a parameter-to-state lookup map as `std::unordered_map<float*, AdamState> _state_by_param_ptr`[14], that provides average constant-time $\mathcal{O}(1)$ lookup complexity. Since weight and bias arrays are allocated once and reside in contiguous memory locations throughout the lifetime of the network, their raw memory addresses (`float*`) serve as unique keys. This pointer-based hashing avoids the overhead of index-based tracking or string-based lookups and enables on-the-fly initialization of the `AdamState` upon the first optimization step.

[13] That accepts predictions, targets and angles of attack (`alphas`) tensors as `const std::shared_ptr<Tensor>&`.

[14] Where `AdamState` is a private structure that contains the first moment vector `std::vector<float> m`, the second moment vector `std::vector<float> v`, and an unsigned integer `timestep` to keep track of the parameter update steps.

CNNModel

The concrete class `CNNModel` coordinates the execution, training and state management of the entire neural network. It aggregates the layers of the convolutional branch and the dense branch, manages feature fusion, interfaces with the optimizer and implements utilities for distributed data-parallel training and weight serialization.

The `forward` method propagates input tensors sequentially through the model. To maximize efficiency, the iteration over the layer containers (`conv_blocks` and `dense_blocks`) utilizes `const auto&`, thereby avoiding the performance overhead associated with incrementing reference counters on the underlying shared pointers. Conversely, the `backward` method employs reverse iterators to correctly propagate gradients upstream in strict accordance with the chain rule.

Another interesting detail is that in the `update` method gradient values are clipped to the interval $[-1.0, 1.0]$ to prevent gradient explosion[15]. This is executed via a local lambda function (`clip_layer`) containing a nested `clip_tensor` lambda, which directly modifies the elements in-place using standard pointer-based loops.

Whenever layer-specific parameters must be accessed for weight updates, gradient synchronization, or serialization, the model dynamically resolves the abstract base pointer types through runtime downcasting via `std::dynamic_pointer_cast` (e.g., casting from `std::shared_ptr<Layer>` to `std::shared_ptr<Conv2DLayer>`).

Finally, the model implements binary weight export and import methods using `std::ofstream` and `std::ifstream`, opened with the `std::ios::binary` flag. Binary data is written and read directly via type-unsafe low-level operations utilizing `reinterpret_cast<const char*>` and `reinterpret_cast<char*>` to map raw byte buffers, which minimizes I/O latency. During the import phase, after reading the shapes and element counts from the file, weight vectors are populated using `std::move` (from the `<utility>` header) to transfer the ownership of heap-allocated arrays into the target structures, bypassing expensive memory allocation and copying. To illustrate this efficient binary format, Table 4 details the layout of the file header and the metadata corresponding to the initial `Conv2DLayer`.

Table 4: Example layout of the binary weight file header, showing the global count of layers followed by the name and weight/bias tensor metadata of the first layer (`Conv2DLayer`).

Byte Offset	Field Name	Value / Example
`0x00-0x07`	Total layers in model (N_{layers})	`8`
`0x08-0x0F`	Layer name length (L_{name})	`39`
`0x10-0x36`	Layer name string	`"Conv2DLayer(1,8,5, stride=5, padding=0)"`
`0x37`	Weights presence flag	`true` (`0x01`)
`0x38-0x3F`	Weights tensor rank (R_W)	`4`
`0x40-0x5F`	Weights tensor shape	`[8, 1, 5, 5]`
`0x60-0x67`	Total weights count (N_W)	`200`
`0x68-0x387`	Weights raw data	`[-0.0125, 0.0841, ...]`
`0x388`	Biases presence flag	`true` (`0x01`)
`0x389-0x390`	Biases tensor rank (R_B)	`1`
`0x391-0x398`	Biases tensor shape	`[8]`
`0x399-0x3A0`	Total biases count (N_B)	`8`
`0x3A1-0x3C0`	Biases raw data	`[0.0, 0.0, ...]`

3.2.6. Data Pipelines: `NPZDataLoader`

The `NPZDataLoader` class manages the retrieval, normalization, and partitioning of datasets stored in the compressed NumPy (`.npz`) format. This format was explicitly selected because it packages multiple named multidimensional arrays within a single ZIP archive, acting as an efficient bridge between Python-based preprocessing scripts (such as `build_dataset.py`) and the C++ training pipeline without the severe parsing overhead

[15] This choice has been made after training the model for the first time, where we have noticed very strong fluctuations of the loss value during the training process.

of text-based formats like CSV.

Specifically, the dataset is structured into four core arrays:
- `X_sdf`, representing the 150×150 SDF of the airfoil geometries;
- `X_scalars`, storing the Reynolds number (Re) and the angle of attack (α);
- `Y_cl`, containing the ground-truth lift coefficient (C_L) labels computed from the Navier-Stokes simulations;
- `metadata`, containing the specific NACA airfoil profile labels.

This class provides a complete input pipeline, converting these raw data arrays into normalized, formatted tensor batches for network training.

To import the data without introducing heavy external C++ NumPy parsing libraries, the internal reading method establishes a read-only POSIX pipe to execute a brief Python script, that utilizes Python's native numpy library to load the specified array, converts it to raw single-precision bytes and streams the output directly to the standard output. The C++ program then intercepts this byte stream, reading it sequentially into a standard vector buffer before safely closing the process and verifying its exit status.

Once the arrays are loaded into memory, the constructor applies Z-score standardization across the SDF, scalar parameters and target lift coefficients, to prevent numerical instability during the training phase. Then, the class handles the sequential feeding of this data during training. The batching process partitions the normalized dataset into mini-batches by maintaining an internal cursor that tracks the current read position. For each requested batch, it allocates new tensor objects and utilizes standard library copying algorithms to transfer the exact slice of data from the internal buffers to the newly created tensors.

3.2.7. Main execution of the CNN

The main execution entry point of the neural network workflow is defined in `main.cpp`. It coordinates the loading of datasets, the structural composition of the CNN layers and the training and validation loops over successive epochs.

The program begins by instantiating the training and testing pipelines using `NPZDataLoader` to load the compressed NumPy datasets. It then creates a shared instance of the `AdamOptimizer` wrapped in a `std::shared_ptr` via `std::make_shared` with a designated learning rate of 10^{-5}. This optimizer is passed to initialize the `CNNModel` instance.

The architecture of the neural network is composed using the model's builder methods:

1. the convolutional branch is constructed by adding a `Conv2DLayer` with 1 input channel, 8 output channels, a 5×5 kernel, and a stride of 5, followed by a `LeakyReLULayer` (with an activation slope $\alpha = 0.05$) and a `FlattenLayer` to reduce the spatial feature maps to a one-dimensional representation of size 7200;

2. the feature fusion stage is enabled by registering a `ConcatenateLayer` via `set_concatenate_layer()`, which merges the geometry features with the two physical flow parameters (Reynolds number and angle of attack);

3. the fully-connected regression head is built using three sequential `DenseLayer`s of dimensions $7202 \to 128$, $128 \to 64$, and $64 \to 1$, separated by `LeakyReLULayer` activations, resulting in a single scalar prediction representing the lift coefficient (C_L).

The training process iterates for 100 epochs. Within each epoch, it resets the data loaders and processes the dataset in mini-batches. For each batch, it invokes `train_step` on the model instance to run the forward pass, evaluate the PINN loss function, compute parameter gradients via backpropagation and update the weights. After the training phase of each epoch, it runs a validation pass by propagating the validation samples through the network via `forward()` to monitor generalization and check for overfitting by computing the MSE against the validation targets.

3.3. Parallelism

To achieve high-performance capabilities and scale the computational workflow to large-scale grids and datasets, the codebase employs distributed-memory parallelism using the **Message Passing Interface (MPI)** standard.

The parallelization strategy is implemented at two distinct stages: the geometric preprocessing phase (for parallel SDF generation) and the neural network training phase (for distributed data-parallel training).

3.3.1. Parallel SDF Generation

The computation of the SDF, as described in Section 3.1.1, is an embarrassingly parallel task, since the evaluation of the signed distance for each point on the 150×150 grid is independent of all other points.

To do this, upon initializing the MPI environment via `MPI_Init`, the total number of grid points ($N_{\text{total}} = 22,500$) is partioned by the communicator size (S, retrieved using `MPI_Comm_size`). Because there could be some cases where N_{total} is not perfectly divisible by S, the remainder ($R = N_{\text{total}} \bmod S$) is distributed among the first R processes[16]. Each process (with rank ID r, retrieved via `MPI_Comm_rank`) computes its local count and offset as:

$$N_{\text{local}} = \lfloor N_{\text{total}}/S \rfloor + \mathbb{I}(r < R) \quad \text{and} \quad O_{\text{local}} = r\lfloor N_{\text{total}}/S \rfloor + \min(r, R) \tag{44}$$

where $\mathbb{I}$ is the indicator function.

Each process independently computes the SDF values for its assigned local grid points. To reconstruct the complete 2D matrix, the local chunks must be gathered into a global vector of size N_{total}, using `MPI_Allgatherv`, which performs a vector gather-to-all communication. The collective call requires `recvcounts`, which stores the number of elements expected from each rank, and `displs`, which specifies the starting byte displacement of each segment in the global buffer. `MPI_Allgatherv` ensures that all processes receive the full SDF image.

Finally, the root process (rank 0) writes the complete matrix to disk via the `mappingToMatrix` method, while other ranks proceed to the next file or terminate safely via `MPI_Finalize`.

3.3.2. Distributed Data-Parallel Training

The training of the Convolutional Neural Network is parallelized in `CNN/main.cpp` and `CNNModel.cpp` using a data-parallel training strategy to scale to large datasets and decrease training epoch time.

In `CNN/main.cpp`, data parallelism is implemented by distributing the training batch across multiple processes. The global mini-batch size is scaled according to $B_{\text{global}} = B_{\text{local}} \times S$, with a local batch size of $B_{\text{local}} = 64$. To prevent redundant memory duplication, each process uses a custom `slice_batch_tensor` method to extract only its specific segment of the global dataset, beginning at the index offset $r \times B_{\text{local}}$.

To guarantee consistent optimization across the distributed environment, the training pipeline enforces MPI synchronization at three critical stages:

1. initially, the root process generates the network weights and distributes them to all other ranks using `MPI_Bcast` on the raw tensor buffers, ensuring an identical starting state[17] across the entire communicator;

2. during the backpropagation phase, each process computes gradients based on its local batch slice. Before applying the Adam optimization step, these local gradients are aggregated via an in-place `MPI_Allreduce` operation. By utilizing the `MPI_SUM` operator directly on the existing memory buffers, the system efficiently avoids redundant data allocations. The accumulated gradients are then divided by the total number of processes S to compute a global average, which guarantees that every rank applies the exact same mathematical update to its local parameters;

3. finally, at the conclusion of each epoch, the local training losses, validation losses, and processed sample counts are gathered back to the root process via `MPI_Reduce`, allowing it to calculate and report the consolidated global statistics.

3.4. Results

Given the significant model size of 930,513 trainable parameters, performing the training process on standard personal computers was computationally impractical within a reasonable timeframe. To overcome this lim-

[16] This partitioning guarantees perfect load balancing, as the workload of any two processes differs by at most a single grid point.

[17] With all the weights initialized to the same value.

itation, the training was conducted on the high-performance computing (HPC) resources of the **CINECA Leonardo cluster**. Specifically, we utilized a full single node configured with two Intel Xeon Platinum 8480+ (Sapphire Rapids) processors, providing a total of 112 physical cores, backed by 512 GiB of DDR5 RAM operating at 4800 MHz. This hardware acceleration reduced the total execution time from several hours to approximately five minutes.

```
#!/bin/bash
#SBATCH --job-name=cnn_training
#SBATCH --account=EUHPC_D35_025
#SBATCH --partition=dcgp_usr_prod
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=112
#SBATCH --time=01:00:00
#SBATCH --output=cnn_%j.out
#SBATCH --error=cnn_%j.err


cd /leonardo/home/userexternal/gdonnine/CNN/
   Ice_acceleration_model_on_airplane_wing/CNN

module purge
module load python/3.11.7
module load gcc/11.3.0
module load intel-oneapi-compilers
module load intel-oneapi-mpi

srun ./cnn_mpi_executable
```

Listing 5: SLURM run script for CNN training (`run_cnn.sh`).

The network was trained for 100 epochs on a synthetically generated dataset of 2D SDF, representing five **symmetric NACA airfoil profiles** (NACA 0006, 0008, 0010, 0012 and 0018) across varying angles of attack (α) and Reynolds numbers (Re). Partitioned via an 80/20 split, the dataset yielded 1,713 training samples and 429 validation samples[18].

Initial Unstable Training Run

In our first training attempt, we adopted a standard learning rate of 10^{-3}, without applying input feature normalization or gradient clipping. This configuration resulted in severe numerical instability during the optimization process. As illustrated in Figure 10, the training and validation losses exhibit chaotic, erratic oscillations, fluctuating wildly by over seven orders of magnitude (ranging from 10^{-1} to 10^{6} in log scale). The presence of massive loss spikes[19] is a clear indicator of exploding gradients. Without gradient clipping, large backpropagated gradients led to excessive parameter updates, pushing the weights into unstable regions of the loss landscape and preventing the optimizer from settling into a local minimum.

Optimized Successful Training Run

To resolve the numerical instability and ensure smooth convergence, three key modifications were implemented in the training pipeline:

1. the Adam optimizer's learning rate (η) was reduced from 10^{-3} to 10^{-5};
2. standard Z-score normalization was applied during the data-loading phase to scale the inputs (SDF and physical scalars) and targets (lift coefficients) to zero-mean and unit-variance;
3. gradient clipping was enabled in the update phase to clamp all gradient components strictly within the interval $[-1.0, 1.0]$ (as detailed in Section 3.2.5).

These modifications successfully stabilized the training trajectory.

[18] All of which have been made publicly available on Kaggle

[19] Such as the prominent peaks around epochs 40, 60, 70 and 85.

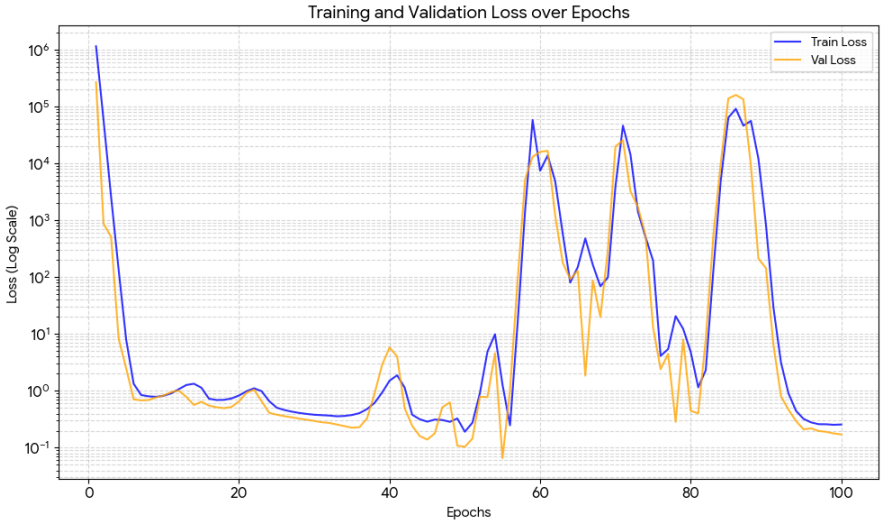


Figure 10: Log-scale evolution of the training and validation losses over 100 epochs during the initial unstable training attempt. The extreme oscillations and sudden spikes up to 10^6 highlight the numerical instability caused by an excessively high learning rate and the absence of gradient clipping or feature normalization.

As shown in Figure 11, the loss curves exhibit highly stable, monotonic convergence. The left plot (Full View) shows the rapid decay of the loss during the initial epochs, indicating that the network quickly learned the dominant physical relationships. The right plot (Zoomed View, Epochs 5-100) highlights the smooth, continuous reduction of both training and validation losses, with no signs of overfitting. The validation loss consistently tracks slightly lower than the training loss; this is a common characteristic when model regularization (such as L2 weight decay) is active during training or when the z-score normalization effectively centers the distribution of test cases.

This optimized training process completed in likely 5 minutes and 11 seconds, achieving a final training loss of 0.0104216 and a validation loss of 0.00784775, demonstrating both the efficiency and accuracy of our implementation.

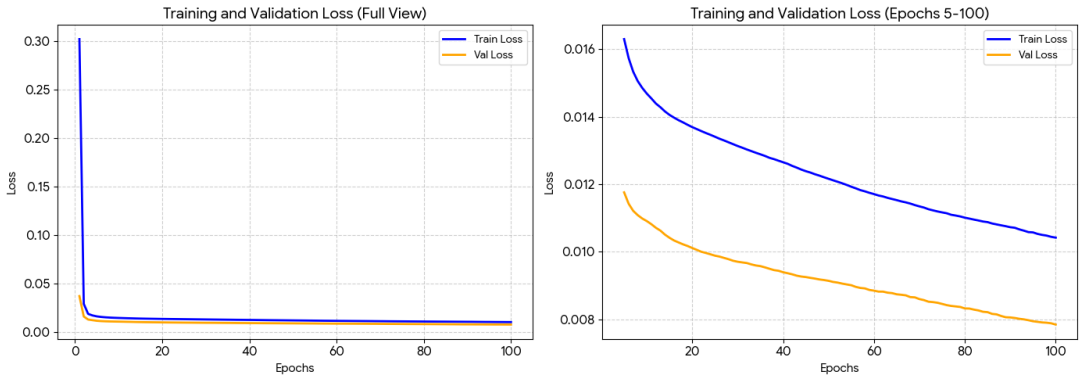


Figure 11: Evolution of training and validation losses for the optimized training run. The full view (left) illustrates the rapid initial convergence, while the zoomed-in view from epoch 5 to 100 (right) highlights the smooth, monotonic decrease of both metrics, confirming the effectiveness of Z-score normalization and gradient clipping.

4. Tuning

The Convolutional Neural Network described and implemented in Chapter 3 provides the starting point for the numerical study presented in this chapter. Although the initial model is already able to approximate the

lift coefficient from the airfoil geometry and the flow parameters, our objective is not limited to obtaining a working implementation. We also want to understand which choices of architecture, optimization algorithm, regularization, activation function, and learning rate allow the surrogate model to obtain more accurate and more reliable predictions. For this reason, we performed a sequence of controlled tuning experiments. In each experiment one group of quantities was varied, while the other settings were kept fixed as far as the corresponding driver allowed. The results were then analysed not only through the final validation error, but also through the evolution of the training objective, the physical validation error, the gradient norms, the actual parameter updates, and the activation distributions.
The order of the study follows the development of the model. First, we investigated the topology of the network, because the feature representation and the capacity of the regression head influence every subsequent experiment. We then considered dropout, the optimizer, the weight of the SIMM physical term, and L1 and L2 regularization. Once these settings had been examined, we compared the available activation functions and finally studied both the magnitude and the schedule of the learning rate. The last part of the chapter describes the long ordinary training run performed with the production executable and collects the conclusions that can be drawn from the complete tuning campaign.

4.1. Cross-validation and diagnostic methodology

All the tuning experiments use the training dataset stored in `dataset/cnn_dataset_train.npz`, which contains 1713 samples. Each sample consists of a 150×150 Signed Distance Function, the scalar pair (Re, α), and the corresponding lift coefficient C_L. The separate file `dataset/cnn_dataset_test.npz`, containing 429 samples, is not used to compare candidates. In the intended procedure, it is loaded only after a candidate has been selected and the selected model has been refitted on the complete training dataset. This separation is important because using test data during the search would turn the final error into a selection quantity rather than an independent estimate.
The comparisons are performed with the project implementation of `RandomKFold`. The samples are shuffled with seed 42 and divided into five folds. The splitter works at sample level, rather than keeping all observations belonging to one airfoil in the same fold. Consequently, the experiment measures interpolation over the sampled dataset and does not constitute a strict test of generalization to an entirely unseen airfoil family. In every fold, approximately eighty percent of the samples are used for training and the remaining twenty percent for validation. In the recorded runs, the validation sets contain 342 or 343 samples and the training sets contain 1370 or 1371 samples. The fold plan is generated once and reused for every candidate within a tuning experiment, so two candidates are compared on exactly the same validation observations.
Before a fold is trained, the normalization statistics are fitted only on that fold's training indices. The same statistics are subsequently applied to the corresponding validation samples. The angle of attack is read in degrees from the scalar input and converted to radians before it is used in the SIMM residual. Every candidate and every fold constructs a fresh model and a fresh optimizer. Therefore, neither the weights, the activation caches, nor the optimizer moments are transferred from another candidate or fold. The distributed training uses a global batch size, so changing the number of MPI ranks changes the distribution of the work but not the nominal batch size or the definition of the metrics.
The primary selection quantity is the physical-unit mean squared error on the validation samples. If $\widehat{C}_{L,i}$ is the predicted coefficient and $C_{L,i}$ is the CFD value, the physical MSE of fold k is

$$\mathrm{MSE}_{\mathrm{val}}^{(k)} = \frac{1}{n_k} \sum_{i \in \mathcal{V}_k} \left(\widehat{C}_{L,i} - C_{L,i} \right)^2 , \tag{45}$$

where $\mathcal{V}_k$ is the validation subset and n_k is its number of samples. The candidate score is the sample-weighted aggregation returned by `CrossValidator`, rather than a comparison of the standardized SIMM objective. This distinction matters because the training objective contains a data term and, in some experiments, a physics term expressed in fold-specific standardized target units, whereas Equation (45) is expressed in the squared physical lift-coefficient units. The fold standard deviation is reported together with the mean to show the sensitivity to the data split.
The training objective is monitored through `train_objective`, while the physical training and validation errors are read from `epoch_metrics.csv`. The public cross-validation history is normally stored every ten epochs, although diagnostics contain a validation value at every epoch. The following diagnostic quantities are used throughout the chapter. The gradient norm is read from rows with `parameter_scope=all`; it is measured after the MPI weighted gradient aggregation and before clipping and the optimizer update. The parameter update ratio is the actual movement of the parameters relative to their norm,

$$r_\theta = \frac{\|\theta_{\mathrm{after}} - \theta_{\mathrm{before}}\|_2}{\max\left(\|\theta_{\mathrm{before}}\|_2, 10^{-12}\right)}, \tag{46}$$

and is not simply a learning rate multiplied by a gradient. It includes the optimizer moments, clipping, decay, the numerical epsilon, and the current parameter values. Finally, activation statistics are distinguished between the pre-activation tensor entering a non-linearity and the post-activation tensor returned by it. Their mean, variance, minimum, maximum, and fixed-bin histograms are used to identify signal attenuation, saturation, or growth of the activation scale. In all cases, we checked the raw CSV files for non-finite values and for late-epoch increases before describing a run as numerically stable.

4.2. Topology tuning

The first tuning activity concerned the topology because the convolutional trunk determines how the 150×150 SDF is compressed, while the dense head determines how the geometric representation is combined with the two scalar flow parameters. A model that is too small may not represent the dependence of C_L on geometry and angle of attack, whereas a model that is unnecessarily large may increase the number of trainable parameters without improving generalization. The experiment therefore varied the convolutional kernel size and the widths and depth of the dense head while keeping the optimizer, loss weight, training budget, and fold plan fixed. This type of structural comparison is consistent with the surrogate model motivation introduced in Chapter 3. The reference to the role of activation and gradient propagation in deep networks discussed by Glorot and Bengio Glorot and Bengio (2010) is kept here only as motivation for studying signal propagation in principle, since that work considers saturating nonlinearities and initialization and does not predict a kernel size or a head width for this SDF trunk.
For a two-dimensional convolution with input size $H_{\text{in}} \times W_{\text{in}}$, kernel size K, stride S, and zero padding P, the output dimensions are

$$H_{\text{out}} = \frac{H_{\text{in}} - K + 2P}{S} + 1, \qquad W_{\text{out}} = \frac{W_{\text{in}} - K + 2P}{S} + 1. \tag{47}$$

The topology experiment used $P = 0$ and set the stride equal to the kernel size, with eight output channels in every candidate. The 5×5 alternative consequently produces eight feature maps of size 30×30, or 7200 flattened features, so that the first dense layer receives 7202 values after concatenation with the two scalar inputs. The 3×3 alternative with stride three produces eight feature maps of size 50×50, or 20000 flattened features, so that the first dense layer receives 20002 values. A head with widths $(w_1, \ldots, w_m)$ then applies successive transformations $\mathbf{h}_{j+1} = \sigma(W_j \mathbf{h}_j + \mathbf{b}_j)$ with LeakyReLU nonlinearity σ and ends with a linear scalar output. Increasing the widths mainly increases the capacity of the first dense transformation, while adding hidden layers changes the depth of the nonlinear mapping. The parameter count makes this distinction concrete. With the 5×5 trunk, the baseline head $(128, 64)$ holds 930305 parameters, the intermediate head $(512, 256)$ holds 3819521 parameters, the selected head $(1024, 512, 256, 128)$ holds 8065025 parameters, and the widest depth two head $(2048, 1024)$ holds 16850945 parameters, where each dense layer contributes $(D_{\text{in}} + 1)D_{\text{out}}$ weights and biases and the convolution contributes $(K^2 + 1) \times 8$ parameters.
The search contained 18 candidates and therefore 90 fold evaluations. Each fold was trained for 100 epochs with global batch size 64, Adam at learning rate 10^{-5}, SIMM physics weight 0.25, LeakyReLU with negative slope 0.05, and seed 42. The baseline was `conv5x5-dense-128-64`. Every candidate was evaluated on the same five fold plan generated once by `RandomKFold` with shuffling and seed 42, using approximately 1370 or 1371 training samples and 342 or 343 validation samples per fold from the 1713 training samples. Normalization was fitted on each fold training split and applied unchanged to the corresponding validation split, and every candidate and fold constructed a fresh model and a fresh optimizer so that no weights or moments were shared. The global batch size does not scale with the number of MPI ranks. The candidate selection was based on the sample-weighted physical validation MSE defined in Equation (45), and the untouched test set of 429 samples was evaluated only after refitting the winning topology on all 1713 training samples. The committed measurements are the CINECA Leonardo log `run_leonardo_51807287.log` together with `results/cross_validation/layer_tuning/summary.csv` and `results/cross_validation/layer_tuning/aggregated.c`
The experiment driver leaves training diagnostics at their default disabled state, so no per epoch `epoch_metrics.csv`, gradient norm, activation statistic, histogram, or update ratio files exist for this grid. The only convergence histories are the public ten epoch checkpoints printed to the log.
The lowest observed validation MSE was obtained by `conv5x5-dense-1024-512-256-128`, with 0.004560 ± 0.000946. It improved on the baseline in all five folds, with per fold values 0.00462266, 0.00618042, 0.00419711, 0.00325115, and 0.00454248 against baseline values 0.00580444, 0.00735140, 0.00622298, 0.00479195, and 0.00587477, for a mean paired difference of -0.001450. The refit of this topology on the complete training set produced an untouched test MSE of 0.002805. Figure 12 shows the ranked means with their fold standard deviations, where the deviation bars describe spread across the five validation splits rather than uncertainty of the mean. The result is not explained by depth alone. At entry width $(512, 256)$, increasing the head from two to four hidden layers reduced the mean error from 0.005052 to 0.004872, while at entry width $(1024, 512)$ the corresponding

Candidate	Mean validation MSE	Fold standard deviation
`conv5x5-dense-1024-512-256-128`	0.004560	0.000946
`conv5x5-dense-1024-512-256-128-64`	0.004626	0.000918
`conv5x5-dense-1024-512`	0.004851	0.000911
`conv5x5-dense-512-256-128-64`	0.004872	0.001125
`conv5x5-dense-512-256`	0.005052	0.001067
`conv5x5-dense-1536-768`	0.005307	0.001177
`conv5x5-dense-2048-1024`	0.005673	0.001466
`conv5x5-dense-128-64` (baseline)	0.006010	0.000823
`conv3x3-dense-128-64`	0.006330	0.001099
`conv5x5-dense-64-32`	0.006425	0.001292
`conv3x3-dense-512-256`	0.006459	0.001318

Table 5: Representative topology candidates in ranked order. The ranking was performed over the complete 18 candidate grid by sample-weighted mean physical validation MSE. The omitted intermediate candidates fall between the rows shown and do not change the conclusions.

increase in depth reduced it from 0.004851 to 0.004560. Increasing width also helped at fixed depth, but the gains became smaller and then reversed. The depth two width sequence runs from 0.006425 at $(64, 32)$ through 0.006010 at the baseline, 0.005447 at $(256, 128)$, 0.005052 at $(512, 256)$, 0.004932 at $(768, 384)$, and 0.004851 at $(1024, 512)$, before rising to 0.005307 at $(1536, 768)$ and 0.005673 at $(2048, 1024)$. The selected depth is therefore interior to the tested depth range, since the depth five extension to `conv5x5-dense-1024-512-256-128-64` is worse at 0.004626, while its first layer width is at the upper boundary of the tested depth four family. It is more accurate to describe the choice as the result of a moderate interaction between capacity and depth than to attribute it to one of these quantities alone, because the width step and the depth step contribute comparable paired improvements of approximately -0.000201 and -0.000292 from $(512, 256)$ toward the winner.

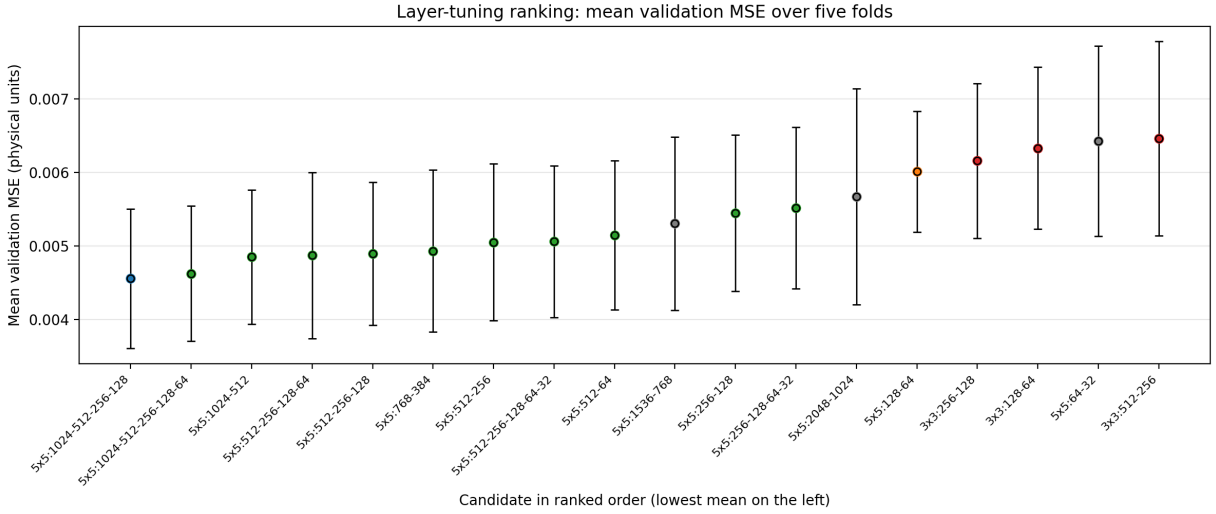


Figure 12: Layer tuning ranking by sample-weighted mean physical validation MSE over five folds. Error bars show the fold standard deviation. The leftmost candidate is the selected `conv5x5-dense-1024-512-256-128`. The plot summarizes `summary.csv` and does not imply a fitted scaling law.

The kernel comparison gives a second clear result. The 5×5 convolution was better than the corresponding 3×3 convolution in each paired comparison available in the grid. For the $(128, 64)$, $(256, 128)$, and $(512, 256)$ heads, the differences in sample-weighted mean MSE were approximately 0.000321, 0.000710, and 0.001407, respectively, always favouring the larger kernel. Figure 13 displays these three pairs with their five per fold values, so the reader can see that the gap widens with head width and that no 3×3 candidate beats the baseline. This behaviour is consistent with the larger receptive field being useful for the SDF representation at the tested stride, although the experiment does not establish that a 5×5 kernel is universally preferable. Figure 14 shows the fold averaged validation MSE at the public ten epoch checkpoints for the winner, the same entry width at

depth two, the baseline, and the 3×3 mate at $(512, 256)$. The two wide candidates separate from the smaller and the 3×3 candidates early and remain lower through epoch 100, while the winner stays slightly below its depth two counterpart after epoch 30. The training objective values printed alongside these checkpoints live in standardized SIMM units and are therefore not plotted on the same axis as the physical MSE.

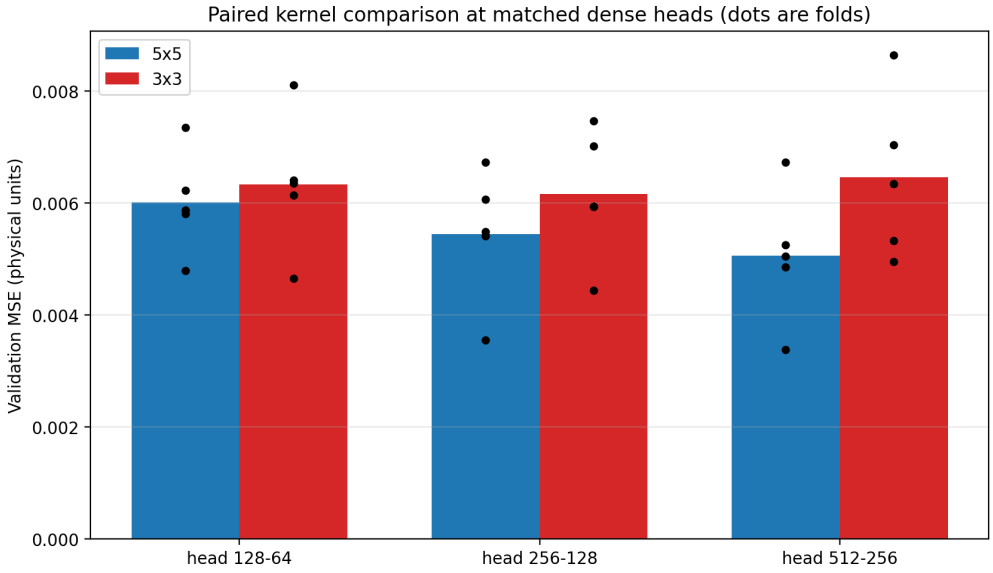


Figure 13: Paired kernel comparison at three matched dense heads. Bars show the sample-weighted mean physical validation MSE and dots show the five per fold values from `aggregated.csv`. All three pairs favour 5×5 with stride five over 3×3 with stride three at zero padding.

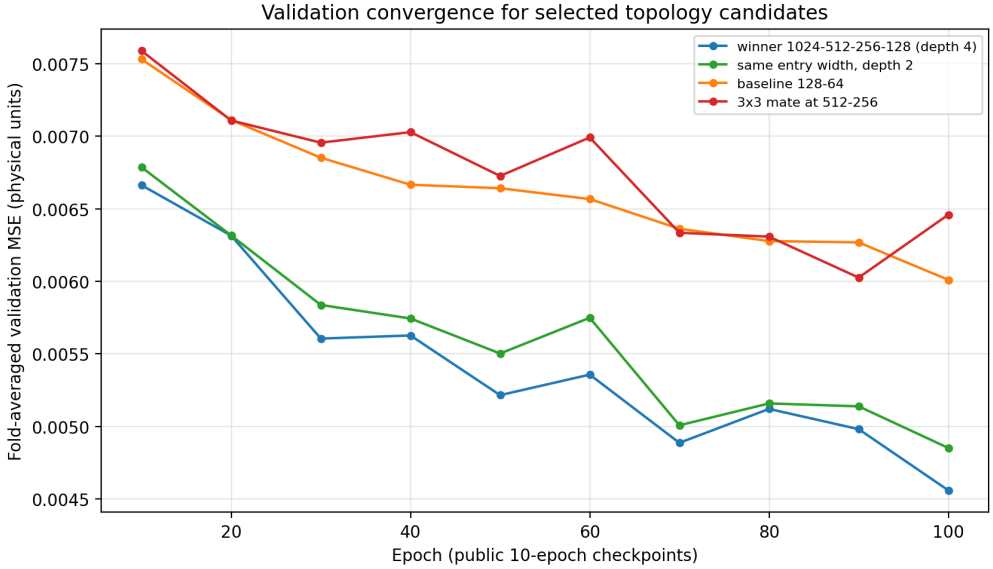


Figure 14: Fold averaged physical validation MSE at public checkpoints every ten epochs, parsed from `run_leonardo_51807287.log`. Curves are shown for the winner `conv5x5-dense-1024-512-256-128`, the depth two candidate `conv5x5-dense-1024-512`, the baseline `conv5x5-dense-128-64`, and the 3×3 mate `conv3x3-dense-512-256`. The training objective is omitted because it uses different standardized units.

The diagnostic conclusion for this grid is limited to the available artifacts. All printed training objectives and validation MSE values are finite, and none of the winner histories shows a late epoch increase that would indicate divergence within the 100 epoch budget. However, gradient norms, actual parameter update ratios, pre-activation and post-activation distributions, variances, and histograms were not recorded for this experiment, so no claim about vanishing or exploding gradients or about activation saturation can be supported for these candidates. The evidence also has three structural limitations. First, each candidate was trained once per fold

with seed 42, so architecture differences cannot be separated from training seed noise. Second, the splitter works at sample level, so the comparison measures interpolation over the sampled dataset rather than generalization to an unseen airfoil family. Third, the selected first layer width sits on the boundary of the tested depth four family and the absolute validation values are optimistic under the random split documented in the experiment README, in which the same profile and angle can appear in both sets with a different Reynolds number. The observed reversal of the widest depth two heads supports the selected configuration, but a new grid with wider and deeper heads would be required to establish behaviour beyond it.
Cross-validation winner: `conv5x5-dense-1024-512-256-128` with physical validation MSE 0.004560 ± 0.000946 and untouched test MSE 0.002805 after refit. Production configuration: the ordinary model built by `make_single_trial` in `CNN/main.cpp` uses a 5×5 convolution with eight channels and stride five, LeakyReLU with $\alpha = 0.05$, and the smaller dense head $(128, 64, 1)$, with source defaults Adam learning rate 10^{-3}, physics weight 0.10, no dropout, no L1 or L2 penalty, gradient clipping at 1, batch size 257, and seed 42. The production choice therefore differs from the topology winner in head capacity, learning rate, physics weight, and batch size. To conclude, we retain the topology winner as the lowest observed candidate of this grid, while the numbers reported for the final long training run in Section 4.9 describe the recorded production configuration and must not be read as a test score for the large topology winner.

4.3. Dropout tuning

Dropout was considered as a stochastic regularizer for the dense regression head. This choice was motivated by the possibility that a head with many trainable parameters could fit the sampled CFD data more closely than it generalizes to the validation samples. The original dropout formulation addresses this situation by randomly removing units during training, thereby reducing excessive co-adaptation, and uses a single unthinned network at inference Srivastava et al. (2014). The preceding regularization experiment, however, showed only a small training to validation gap relative to the fold variability. The dropout experiment therefore tests a specific question: can stochastic masking improve validation accuracy when the available evidence does not indicate a large deterministic overfitting gap?
During training, an activation is multiplied by a Bernoulli mask and, with inverted dropout, the surviving values are scaled by the reciprocal of the survival probability. If z_j is a hidden activation, the training value is

$$\widetilde{z}_j = \frac{m_j}{1-p} z_j, \qquad m_j \sim \text{Bernoulli}(1-p), \tag{48}$$

where p is the probability of dropping an element and $1 - p$ is the survival probability. The same scaled mask is used during backpropagation, so that

$$\frac{\partial L}{\partial z_j} = \frac{m_j}{1-p} \frac{\partial L}{\partial \widetilde{z}_j}. \tag{49}$$

At inference, the mask is disabled and the layer is the identity, so no additional rescaling is needed. The implementation follows this rule in `DropoutLayer`: a zero rate also takes the identity path, while nonzero rates store the scaled mask for the backward pass. The layer has no trainable parameters. In the current experiment source, `make_blueprint` inserts it after every hidden `LeakyReLULayer`, and before the next `DenseLayer`; it is not applied after the final scalar output. The tested grid was $p \in \{0, 0.1, 0.2, 0.3, 0.5\}$.
The mask is generated differently from a sequential random number stream. In the implementation, the trainer combines the fold seed, epoch, and global batch offset into a stream seed; `DropoutLayer` then combines this value with a layer-specific salt, the global sample row, and the feature index through the stateless `splitmix64` helper. The sample row is measured relative to the global batch, not the rank-local slice. Consequently, the mask does not depend on the number of MPI ranks, while different dropout layers receive decorrelated masks. This is relevant to the paired comparison because the dropout recipe is present in every candidate, including $p = 0$, where it is an identity layer. The candidates therefore have the same ordered layer structure, fold seeds, and initial trainable parameters within this sweep. The absolute scores should not be compared with an unrelated no-dropout run whose model recipe omits those identity layers, because that would change the per-recipe seed sequence.
The recorded run evaluated five candidates on the 1713 sample training dataset, giving 25 fold evaluations. The five folds were generated by `RandomKFold` with shuffling and seed 42, and each fold contained 1370 or 1371 training samples and 342 or 343 validation samples. The normalization statistics were fitted on the training indices of each fold only. The fixed network used the convolutional trunk, LeakyReLU with $\alpha = 0.05$, scalar concatenation, and a dense regression head. Adam used learning rate 10^{-5} with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, and zero weight decay. The SIMM physics weight was 0.25, L1 and L2 weights were zero, the global batch size was 64, element-wise gradient clipping used a threshold of 1, and the training seed was 42. Each fold ran for 100 epochs without early stopping. The recorded metadata gives 16 MPI ranks, 22 global optimizer batches per epoch, and therefore 2200 optimizer steps per fold. The effective learning rate in `epoch_metrics.csv` was

constant at 9.9999997×10^{-6}, and `learning_rate_steps.csv` was empty, so this run did not use a schedule. The test dataset was not loaded by the sweep driver and remained outside candidate selection.
Candidate selection used the sample-weighted physical-unit validation MSE returned by `CrossValidator`. The official aggregate file is `cv_summary.csv`; the paired differences, final training MSE, and training to validation gaps are read from `sweep_dropout.csv`. The two files differ slightly in their arithmetic means because the official score weights the 342 and 343 sample validation folds, while the sweep CSV is also used to preserve the fold-by-fold paired inputs. Table 6 reports the complete grid. The last three columns are descriptive quantities and do not replace the physical validation MSE as the selection metric.

p	Mean val. MSE	Fold std.	Paired Δ	Mean train MSE	Val. − train
0.0	0.005834	0.001007	0.000000	0.005421	+0.000413
0.1	0.006686	0.001060	+0.000852	0.006390	+0.000296
0.2	0.007630	0.000861	+0.001796	0.007236	+0.000395
0.3	0.008467	0.001141	+0.002632	0.008098	+0.000369
0.5	0.011764	0.002603	+0.005930	0.011354	+0.000411

Table 6: Complete recorded dropout sweep. Mean validation MSE and fold standard deviation are the sample-weighted values from `cv_summary.csv`. Paired differences, training MSE, and validation minus training gaps are calculated from `sweep_dropout.csv`; a positive paired difference means that the rate is worse than the $p = 0$ reference.

The rate $p = 0$ had the lowest observed physical validation MSE, $0.005833893 \pm 0.001006787$. The ranking then increased with every tested nonzero rate: $p = 0.1$ reached $0.006686315 \pm 0.001059778$, $p = 0.2$ reached $0.007630451 \pm 0.000860791$, $p = 0.3$ reached $0.008466639 \pm 0.001140810$, and $p = 0.5$ reached $0.011763615 \pm 0.002603010$. The rate $p = 0.1$ is the runner-up by the official mean, but it is worse than the reference in all five folds. The reference fold values were 0.0040690648, 0.0054384293, 0.0067913457, 0.0061637875, and 0.0067103554, whereas the $p = 0.1$ values were 0.0047722511, 0.0068476000, 0.0080367438, 0.0068024892, and 0.0069736654. The strongest rate was also worse in all folds, with values 0.0092481814, 0.0116964081, 0.0120545515, 0.0093871192, and 0.0164385148. The last value, from zero-based fold 4, is the largest validation error in the sweep and accounts for much of the $p = 0.5$ fold dispersion.
Figure 15 shows the official ranking and its fold deviations. Figure 16 shows the paired differences $\mathrm{MSE}_{\mathrm{val}}(p) - \mathrm{MSE}_{\mathrm{val}}(0)$ for every fold. The mean paired differences for $p = 0.1$, 0.2, 0.3, and 0.5 were, respectively, $+0.000851953 \pm 0.000468881$, $+0.001795869 \pm 0.000575313$, $+0.002632161 \pm 0.000879863$, and $+0.005930358 \pm 0.002391029$, where the deviations are the sample standard deviations across folds. No nonzero candidate improved on the reference in any fold. Thus, none satisfies the pre-registered rule of improving in at least four of five folds while having a negative mean paired difference whose magnitude exceeds its fold deviation. The paired analysis and the official cross-validation ranking give the same selected rate, but the paired rule is not being used to replace the requested MSE criterion.
The training to validation gaps provide the corresponding generalization check. For $p = 0$, the mean training MSE was 0.005421198 and the mean validation minus training gap was +0.000413398, smaller than the official fold deviation of 0.001006787. The $p = 0.1$ rate reduced the gap to +0.000296338, but its validation MSE increased by approximately 0.000852; the reduction in the gap was therefore not useful. At $p = 0.5$, the gap returned to +0.000411055 while the mean training MSE rose to 0.011353900 and the validation MSE approximately doubled relative to the reference. This pattern is consistent with dropout adding optimization noise and reducing the amount of fitting achieved in the fixed 100-epoch budget, rather than correcting a substantial overfitting effect. It is an interpretation of this fixed experiment, not evidence that dropout cannot help under a different head, training budget, or dataset split.
The convergence diagnostics support the performance ranking without indicating numerical divergence. The recorded `epoch_metrics.csv` files contain one training SIMM objective value per epoch and validation physical MSE at the ten-epoch checkpoints used by this run. The quantities must be kept on separate scales and are not subtracted. Averaged over folds, the $p = 0$ validation checkpoint mean decreased from 0.0071253 at epoch 10 to 0.0058346 at epoch 100. The corresponding $p = 0.5$ values were 0.0227140 and 0.0117650, so the strong dropout rate improved during training but remained substantially worse. The curves contain small fold-averaged fluctuations, particularly for the nonzero rates, but none shows a late unbounded increase. The training objective decreases for all candidates, while its final mean is approximately 0.0123 for $p = 0$ and 0.1142 for $p = 0.5$. These objective values are in standardized SIMM units and are not numerical alternatives to the physical validation MSE. Figure 17 presents both quantities with separate axes.
The gradient and parameter diagnostics show why the high error of $p = 0.5$ should not be labelled a divergent run. For the $p = 0$ candidate, the largest recorded `maximum_norm` was 37.841272. It occurred at zero-based fold 3 and epoch 1 among rows with `parameter_scope=all`. The corresponding maximum at epoch 100 was 2.692005, so the largest early gradient did not turn into a late increase. The $p = 0.5$ candidate reached a larger

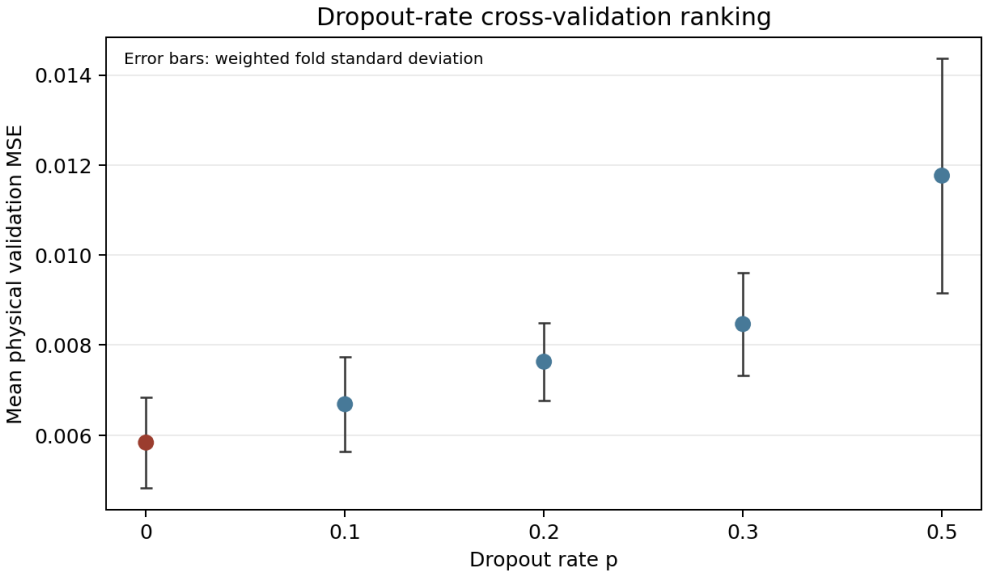


Figure 15: Dropout-rate ranking from the official `cv_summary.csv`. Points are sample-weighted mean physical validation MSE and error bars are the weighted standard deviations over the five folds. The $p = 0$ reference is highlighted in red.

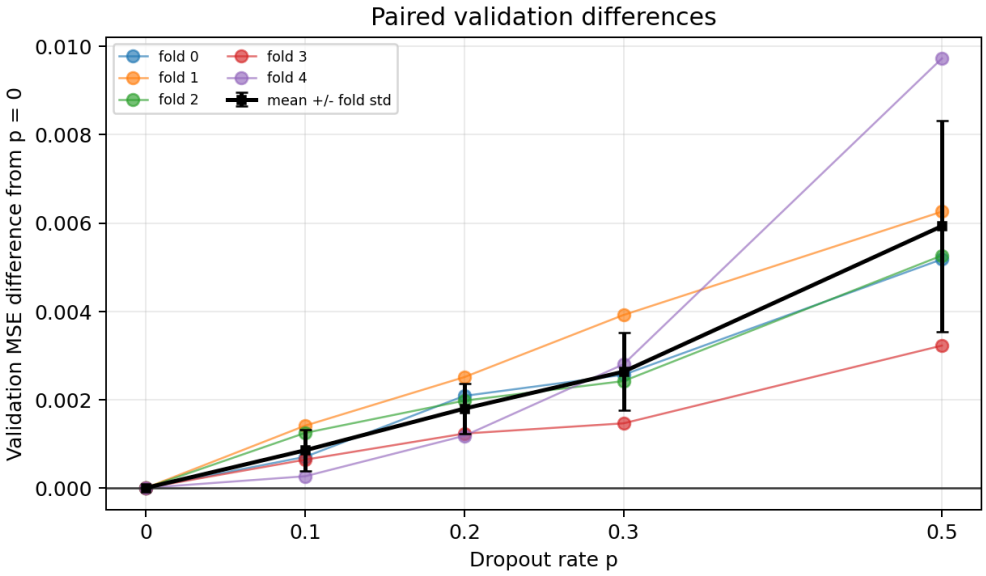


Figure 16: Paired physical validation MSE differences relative to the $p = 0$ reference, computed from `sweep_dropout.csv`. Coloured curves show the five zero-based folds, while black markers and error bars show the mean paired difference and its sample standard deviation. Positive values indicate deterioration relative to the reference.

early maximum of 47.444652 at zero-based fold 4 and epoch 1, while its epoch-100 maximum was 7.286522. All values were finite, so the larger transient is a relative diagnostic difference rather than a failed optimization.
The actual parameter movement was small in both cases. For $p = 0$, the largest `mean_ratio` was 5.0837548×10^{-4}. It occurred at zero-based fold 2 and epoch 1 among rows with `parameter_scope=all`. The corresponding mean pre-update norm was 15.859737 and the mean update norm was 0.00806269. No row reported a near-zero denominator event. For the weight-only scope, the largest pre-update norm for $p = 0$ was 15.878200 at zero-based fold 0, epoch 100, compared with 15.870497 at epoch 1. The largest value for $p = 0.5$ was 15.889940, so the poor validation result cannot be attributed to an unbounded parameter norm. The update-ratio curves in Figure 18 use the actual before and after parameter values, not the learning rate multiplied by a gradient; they include the optimizer state, clipping, and the current parameter scale.
Activation statistics give a compatible but limited view of signal propagation. The rows are globally aggregated

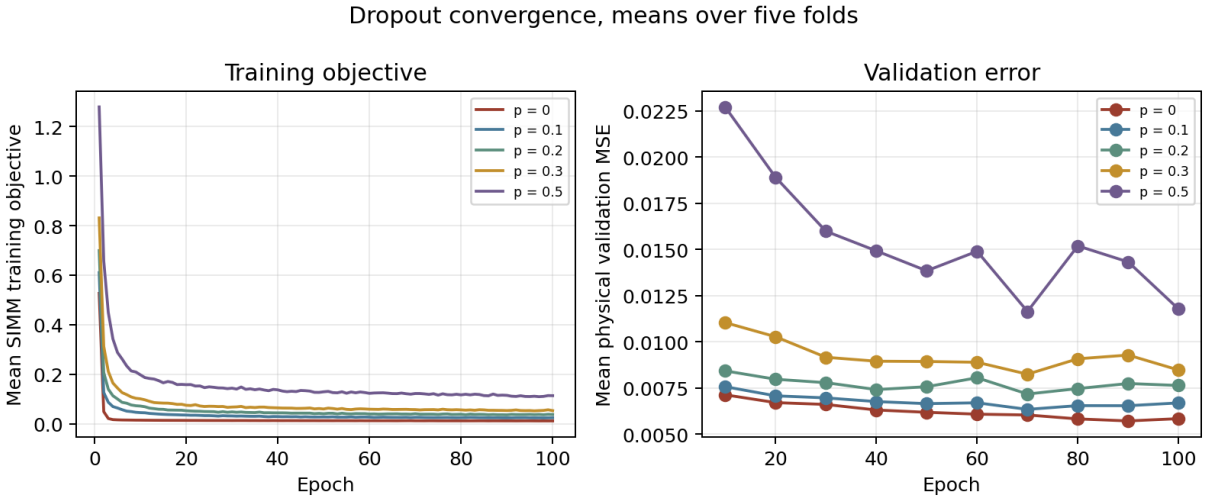


Figure 17: Means over the five recorded folds for the training SIMM objective and the physical validation MSE. The training objective is recorded at every epoch, whereas the validation values in the committed diagnostic files occur at ten-epoch checkpoints. The two panels use separate units, and the curves describe the recorded dropout sweep.

population statistics; dropout itself does not produce activation statistics, and the recorded post-activation values are measured before the subsequent dropout operation. For $p = 0$, the largest post-activation variance over all folds and epochs was 0.186070 at zero-based fold 4 and epoch 100. The corresponding maxima were 0.157550 for $p = 0.1$, 0.153652 for $p = 0.2$, 0.151285 for $p = 0.3$, and 0.185163 for $p = 0.5$. These finite values do not establish that dropout caused signal collapse or saturation. The fixed histograms used 64 bins over $[-10, 10]$ and placed out-of-range values in the edge bins, so the edge counts were not interpreted as precise tail probabilities. Figure 18 aggregates the diagnostic values by dropout rate and shows the peak gradient check for all rates.

The cross-validation winner is therefore `dropout rate = 0.0`, with physical validation MSE $0.005833893 \pm 0.001006787$ on the recorded sweep. The production configuration in `CNN/main.cpp` also uses no dropout by default: `make_single_trial` passes the source default `dropout=0.0` to the regression head. The ordinary production path otherwise differs from this sweep, using source defaults Adam learning rate 10^{-3}, physics weight 0.10, global batch size 257, gradient clipping at 1, and seed 42, with command-line options available to override the rate. Thus, the selected dropout rate agrees with the production configuration. The test set was untouched, so no dropout-specific independent test estimate exists. To conclude, the available evidence supports retaining $p = 0$ for the production model because every nonzero rate worsened the paired validation result and none reduced the small reference gap enough to compensate for its higher validation error. This conclusion is limited to the recorded data split and training budget.

4.4. Optimizer comparison

The optimizer controls how the gradient is converted into a parameter update. For plain stochastic gradient descent, the update is

$$\theta_{t+1} = \theta_t - \eta g_t, \tag{50}$$

where g_t is the synchronized mini-batch gradient and η is the learning rate. Momentum introduces an accumulated direction, while AdaGrad uses the cumulative squared gradients to construct a coordinate-wise scale. RMSprop replaces the cumulative sum with an exponential average. Adam combines exponential first and second moments and applies bias correction Duchi et al. (2011); Kingma and Ba (2015):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \tag{51}$$

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \widehat{v}_t = \frac{v_t}{1 - \beta_2^t}, \qquad \theta_{t+1} = \theta_t - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}. \tag{52}$$

The experiment compared the five optimizers implemented in the project at the common learning rate 10^{-5}, with momentum 0.9 for SGD with momentum, decay 0.9 for RMSprop, and Adam parameters $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\varepsilon = 10^{-8}$. Every candidate received a fresh optimizer state in every fold.

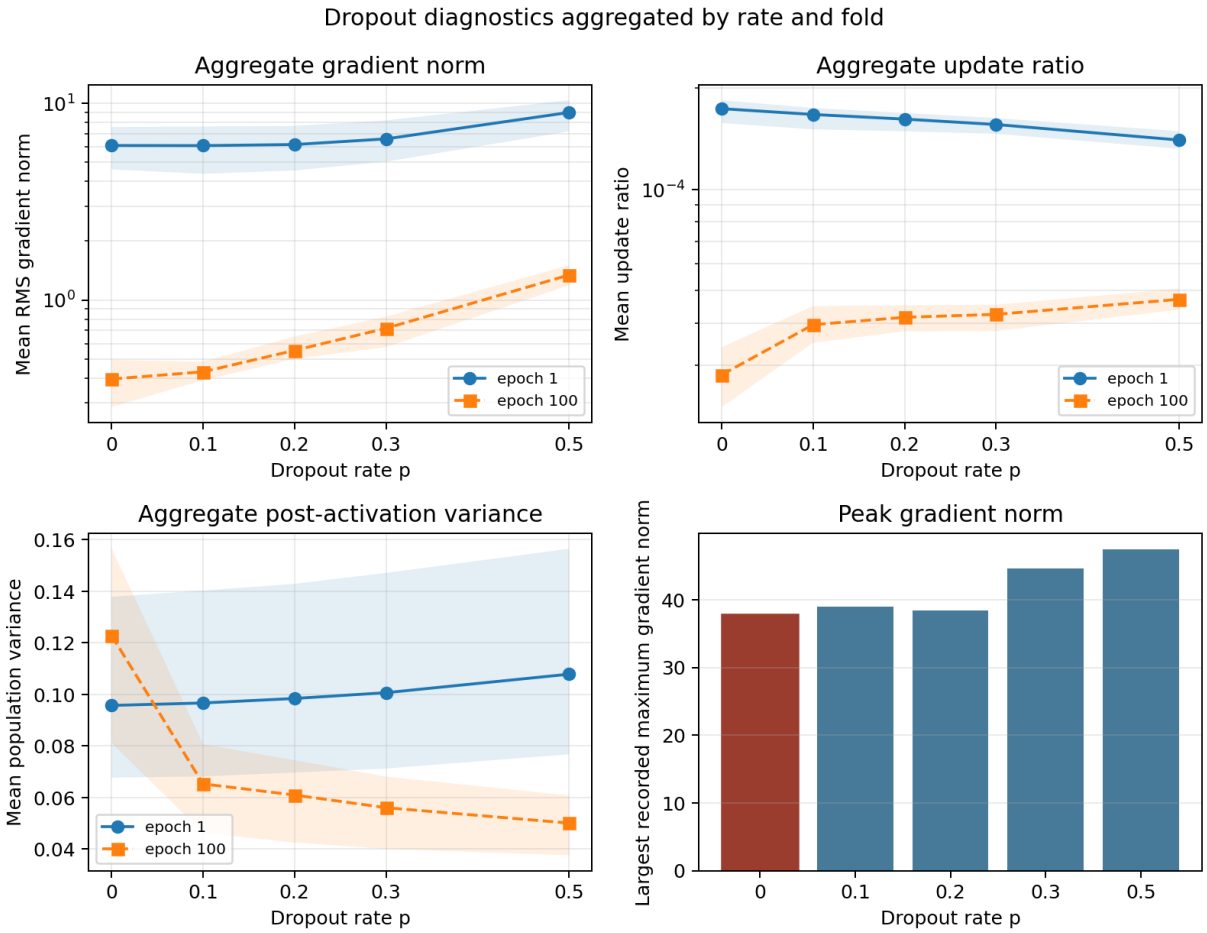


Figure 18: Diagnostics for the recorded dropout sweep. The aggregate gradient norm, actual update ratio, and post-activation variance panels show means with minimum to maximum bands over five folds at epochs 1 and 100; the horizontal axis is dropout rate and the first two vertical axes are logarithmic. The final panel shows the largest recorded `maximum_norm` with `parameter_scope=all` over all folds and epochs for each dropout rate. Diagnostic rows are aggregated and individual layers are not distinguished.

The selected topology `conv5x5-dense-1024-512-256-128` was fixed, together with the SIMM physics weight 0.25, batch size 64, 100 epochs, and seed 42. Adam obtained the lowest validation MSE, as shown in Table 7. The complete ranking is consistent with the expected distinction between adaptive and non-adaptive updates: RMSprop was second, AdaGrad followed it, momentum SGD was worse, and plain SGD was substantially worse at this learning rate.

Optimizer	Mean validation MSE	Fold standard deviation
Adam	0.004427	0.000887
RMSprop	0.005216	0.000653
AdaGrad	0.006185	0.001238
SGD with momentum	0.007151	0.001412
SGD	0.017927	0.006229

Table 7: Cross-validation comparison of the five optimizers at learning rate 10^{-5}.

Adam reduced the mean validation error by approximately 15.1 percent relative to RMSprop and by 38.1 percent relative to momentum SGD. Plain SGD also had the largest fold spread, indicating that its poor mean was accompanied by greater sensitivity to the validation split. The result does not mean that Adam is independent of the learning rate; it only establishes Adam as the best optimizer among the candidates at 10^{-5} under this fixed topology, loss, and training budget. The final refit of the selected optimizer produced a recorded untouched-test MSE of 0.003655. The final diagnostics remained finite, and the actual update ratios were small compared with

the parameter norms, which is consistent with a convergent rather than an unstable trajectory.

4.5. Physics-weight tuning

The physics-informed loss introduced in Chapter 3.1.2 combines the data error with a SIMM residual. In standardized target units, the loss used by the trainer can be written as

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \mathcal{L}_{\text{physics}}, \tag{53}$$

where λ controls the influence of the physical relation. For the symmetric profiles considered in this project, the thin-airfoil relation is $C_L \approx 2\pi\alpha$ in the low-angle regime. The physics residual is masked according to

$$M_i = \begin{cases} 1, & |\alpha_i| \le 10^\circ, \\ 0, & |\alpha_i| > 10^\circ, \end{cases} \qquad \mathcal{L}_{\text{physics}} = \frac{1}{N} \sum_{i=1}^{N} M_i \left(\widehat{C}_{L,i} - 2\pi\alpha_i \right)^2 . \tag{54}$$

The relation is transformed into the fold's standardized target space during training, but the candidate selection remains the physical-unit validation MSE.
The committed experiment tested $\lambda \in \{0, 0.05, 0.1, 0.25, 0.5, 1, 2\}$ with Adam at 10^{-5}, batch size 64, 100 epochs, and five folds. As with dropout, the historical driver used the smaller $(128, 64)$ head. The source has now been corrected to construct the selected large topology, and the provenance of the existing values is recorded in `physics_weight_tuning/README.md` in the result directory. The existing measurements are therefore retained as historical results and are not treated as a new experiment on the corrected topology.

λ	Mean validation MSE	Fold standard deviation
0.00	0.005602	0.001228
0.05	0.005639	0.001270
0.10	0.005630	0.001247
0.25	0.005708	0.001337
0.50	0.006027	0.001334
1.00	0.006583	0.001510
2.00	0.007408	0.001698

Table 8: Historical physics-weight sweep on the earlier $(128, 64)$ dense head.

The lowest recorded error was obtained at $\lambda = 0$. Increasing the weight did not improve either of the two angle groups. For the reference case, the low-angle validation MSE was 0.004097 over 1442 samples and the high-angle MSE was 0.013604 over 271 samples. At $\lambda = 2$, these values increased to 0.005843 and 0.015727, respectively. Thus, the stronger prior did not produce a targeted reduction in the region where it is active. It also increased the peak gradient norm from 56.87 at $\lambda = 0$ to 149.44 at $\lambda = 2$, while all recorded values remained finite. The historical result therefore favours the data-only candidate for this fixed architecture and budget, but it must not be generalized to the corrected large topology until the sweep is rerun.

4.6. Weight regularization

Weight regularization was introduced to test whether reducing parameter magnitude could improve generalization. The two penalties considered in the experiment are applied to weight tensors, not to biases. For a weight vector $\mathbf{w}$, the L1 and L2 contributions can be expressed as

$$\mathcal{L}_{\text{L1}} = \lambda_1 \|\mathbf{w}\|_1, \qquad \mathcal{L}_{\text{L2}} = \lambda_2 \|\mathbf{w}\|_2^2. \tag{55}$$

The L1 term promotes sparsity through a sign-dependent subgradient, whereas the L2 term continuously penalizes large weights. These loss penalties are distinct from optimizer weight decay. The experiment used two independent one-dimensional grids so that the effect of L1 could be separated from the effect of L2.
The large selected topology was used here: `conv5x5-dense-1024-512-256-128`, with Adam at 10^{-5}, physics weight 0.25, batch size 64, five folds, seed 42, and 100 epochs. There were eight L1 candidates and eight L2 candidates. The reference value 0 was included independently on both axes. The available result is a historical reference run preserved in the experiment README rather than a current raw result tree, so the values below are reported with that limitation.

λ_1	Mean validation MSE	Fold standard deviation
0	0.004419	0.000794
6.75×10^{-7}	0.004424	0.000788
2.13×10^{-6}	0.004436	0.000800
6.75×10^{-6}	0.004478	0.000811
2.13×10^{-5}	0.004540	0.000832
6.75×10^{-5}	0.004684	0.000902
2.13×10^{-4}	0.005100	0.000950
6.75×10^{-4}	0.006462	0.001189

Table 9: L1 regularization sweep on the selected topology.

λ_2	Mean validation MSE	Fold standard deviation
0	0.004419	0.000794
10^{-4}	0.004483	0.000819
3.16×10^{-4}	0.004580	0.000859
10^{-3}	0.004768	0.000932
3.16×10^{-3}	0.005092	0.000992
10^{-2}	0.005902	0.001100
3.16×10^{-2}	0.006994	0.001217
10^{-1}	0.008453	0.001080

Table 10: L2 regularization sweep on the selected topology.

In both axes the zero-penalty candidate had the lowest observed validation MSE. The nonzero values degraded the error monotonically, and no candidate met the pre-registered paired criterion of improving at least four of the five folds with a mean paired difference larger than its own spread. The regularization terms were nevertheless active: the sum of squared weights fell from approximately 2993 at zero L2 to 582 at the largest L2 value, while the training MSE increased more rapidly than the validation MSE. This behaviour indicates that the penalties were restricting the fit rather than correcting a substantial overfitting gap. The validation minus training gap decreased, but the improvement was not sufficient to compensate for the loss in training accuracy. The selected configuration is consequently $\lambda_1 = 0$ and $\lambda_2 = 0$ for the tested learning rate and epoch budget.

4.7. Activation-function tuning

The activation function determines the nonlinear transformation and the gradient transmitted between consecutive affine layers. The experiment compared Tanh, Sigmoid, ReLU, and three LeakyReLU slopes. The LeakyReLU function used in the implementation is

$$\sigma(x) = \begin{cases} x, & x > 0, \\ \alpha x, & x \le 0, \end{cases} \qquad \sigma'(x) = \begin{cases} 1, & x > 0, \\ \alpha, & x \le 0. \end{cases} \tag{56}$$

ReLU is recovered when the negative branch is set to zero, whereas Tanh and Sigmoid bound their outputs and can enter saturation at large absolute inputs. The comparison of these effects is relevant to the observed activation variances and gradient norms, and is consistent with the analysis of saturation and layer-wise signal propagation in Glorot and Bengio (2010).

The fixed topology was `conv5x5-dense-1024-512-256-128`. Adam used learning rate 10^{-5}, the physics weight was 0.1, dropout and L1/L2 regularization were zero, and the batch size, epoch count, fold count, and seed were 64, 100, 5, and 42. The six candidates and their measured scores are listed in Table 11.

LeakyReLU with $\alpha = 0.05$ had the lowest mean error and the smallest final fold dispersion. Its peak gradient norm was 20.14 and its peak weight update ratio was 4.50×10^{-4}. All metrics remained finite. The Tanh candidate had a higher peak gradient norm of 111.51, which is consistent with a stronger transient during the early optimization, while Sigmoid showed a much larger mean checkpoint fold standard deviation of 0.02919, consistent with slower and less uniform early convergence. These observations do not by themselves prove that saturation caused the ranking, but they support the interpretation that the selected LeakyReLU provided a more regular signal scale for this model. The final refit of the selected activation achieved an untouched-test physical MSE of 0.002739.

Activation	Mean validation MSE	Fold standard deviation
LeakyReLU, $\alpha = 0.05$	0.004427	0.000865
LeakyReLU, $\alpha = 0.01$	0.004448	0.000876
ReLU	0.004454	0.000876
LeakyReLU, $\alpha = 0.1$	0.004545	0.000966
Tanh	0.005466	0.001045
Sigmoid	0.008971	0.001250

Table 11: Activation-function comparison on the selected topology.

4.8. Learning-rate and schedule tuning

After choosing the topology, optimizer, loss settings, and activation, we studied the learning rate. In Adam, the learning rate multiplies the bias-corrected adaptive direction in Equation (52). It therefore controls the scale of the actual parameter movement even though the denominator adapts to the history of each parameter. The experiment considered constant rates, step decay, cosine annealing, and warm-up followed by cosine decay. For a constant rate, $\eta_e = \eta_0$. For step decay, the implementation uses

$$\eta_e = \eta_0 \gamma^{\lfloor e/S \rfloor}, \tag{57}$$

where S is the step interval and γ is the multiplicative factor. The cosine candidates use

$$\eta_e = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left[1 + \cos\left(\frac{e}{E-1} \pi \right) \right], \tag{58}$$

and the warm-up candidate first increases linearly from 10^{-5} to 10^{-1} over five epochs before applying the cosine decay. These expressions describe the configured rate; the diagnostic file records the effective rate used at each epoch.

The fixed model was `conv5x5-dense-1024-512-256-128`, with Adam, physics weight 0.25, batch size 64, 100 epochs, five folds, and seed 42. Eleven candidates were evaluated. The constant 10^{-3} candidate produced the lowest validation error, while every candidate that reached 10^{-1} diverged.

Candidate	Mean validation MSE	Fold standard deviation
`const_1e-3`	0.003609	0.001335
`step_s15_1e-3`	0.004005	0.000704
`step_s30_1e-3`	0.004072	0.000785
`const_1e-5`	0.004419	0.000790
`step_s30_1e-5`	0.005034	0.000901
`step_s15_1e-5`	0.005094	0.000905
`const_1e-1`	2.13×10^{17}	3.12×10^{17}

Table 12: Representative learning-rate candidates. The complete grid also contained four additional candidates that reached 10^{-1} and diverged.

The selected constant rate 10^{-3} improved the mean validation MSE by approximately 18.3 percent relative to the constant 10^{-5} baseline. The step schedules starting at 10^{-3} were stable, but their early reductions prevented the parameters from continuing to move through the useful part of the loss surface within the 100-epoch budget. The schedules starting at 10^{-5} became even less effective after their reductions. In contrast, all five candidates involving a rate of 10^{-1} produced errors from approximately 2.09×10^8 to 8.31×10^{17}, depending on the schedule and fold. Even the five-epoch warm-up did not avoid the instability associated with the high peak rate. The data therefore support 10^{-3} as the lowest observed rate setting for this search, rather than a universal stability threshold. Its final refit produced an untouched-test MSE of 0.003254.

4.9. Final production training

The final ordinary training was launched through the production path rather than through one of the search drivers. This distinction is necessary because a tuning winner and the configuration used by the production executable are not automatically identical. In the current `CNN/main.cpp`, the ordinary model is built with a

5×5 convolution having eight output channels and stride five, followed by LeakyReLU with $\alpha = 0.05$, flattening, scalar concatenation, and the dense head $(128, 64, 1)$. The production source defaults use Adam with learning rate 10^{-3}, physics weight 0.10, no dropout, no L1 or L2 penalty, gradient clipping at 1, batch size 257, and seed 42. The final SLURM command used 64 MPI ranks and increased the epoch budget to 2500. Early stopping used a validation fraction of 0.10, minimum epoch and patience settings of 20, an overfit ratio of 0.15, and restoration of the best checkpoint.
The selected epoch in the recorded final run was 1505. At that checkpoint, the physical training MSE was 0.0004932 and the validation MSE was 0.0005995. The independent test evaluation over 429 samples gave 0.0006727. The test error was 0.0005468 over the 363 samples with $|\alpha| \leq 10^\circ$ and 0.0013653 over the 66 samples outside that range. The higher error at high angle is consistent with the fact that the SIMM relation is masked there, although the error split alone does not establish the cause. The diagnostic files for this run contained finite metrics, and the training history continued beyond the selected epoch before the best checkpoint was restored.
The final run should not be compared numerically with every cross-validation value without qualification. The tuning experiments use fold-specific normalization and approximately 80 percent of the training samples in each fold, whereas ordinary training uses a 90/10 split and the final refit uses the complete training set before evaluating the test data. In addition, the production source still contains the smaller $(128, 64)$ head, while the topology search selected the larger head. The final numbers are therefore the performance of the recorded production configuration, not a test score for the large topology winner. They are nevertheless the appropriate values for describing the executable that was finally used.

4.10. Conclusions

The tuning campaign shows that the largest improvement among the structural choices came from replacing the initial $(128, 64)$ dense head with the selected topology `conv5x5-dense-1024-512-256-128`. Its cross-validated physical MSE was 0.004560, compared with 0.006010 for the topology baseline, and the improvement was observed in all five folds. Adam was the best optimizer at the tested rate 10^{-5}, with a mean validation MSE of 0.004427. The activation comparison selected LeakyReLU with $\alpha = 0.05$, and the learning-rate search selected a constant Adam rate of 10^{-3}, which reduced the cross-validated error to 0.003609 for that experiment. These choices are supported not only by the mean scores, but also by finite diagnostics, small actual update ratios, and the absence of late-epoch divergence in the selected candidates.
The regularization results were different from what might be expected for a large network. Both dropout and weight penalties increased the recorded validation error, and the physics-weight sweep also selected the zero-weight reference. The dropout and physics-weight values must be read as historical results from the earlier $(128, 64)$ head, because those searches were completed before their C++ drivers were corrected to construct the selected large topology. The mismatch is explicitly documented beside the result trees, and no corrected numerical claim is made without a new run. For the large topology, the L1 and L2 study likewise selected zero penalty, indicating that at the tested learning rate and 100-epoch budget the reduction in the training and weight norms did not compensate for the loss in fitting accuracy.
The final ordinary training produced a substantially lower physical test MSE of 0.0006727 after refitting the production model and restoring its best checkpoint. This value reflects the full training procedure, the longer 2500-epoch budget, the 90/10 ordinary split, and the production defaults in `CNN/main.cpp`; it is not an unbiased test estimate for every tuning candidate. Overall, the experiments demonstrate that the accuracy of the surrogate depends on a coordinated choice of topology, optimizer, activation, and learning rate rather than on adding regularization indiscriminately. They also show why numerical diagnostics and explicit provenance are necessary: a low validation value is meaningful only when the topology, split, metric, and training configuration that produced it are stated alongside the result.

5. Conclusions

This work successfully demonstrates an integrated approach to aerodynamic analysis, combining high-fidelity numerical simulation with a data-driven surrogate model. The primary achievement was the extension of the `LifeX` library to handle turbulent external aerodynamic flows through the integration of the **Spalart-Allmaras RANS model**. This new capability was validated on the **Leonardo HPC cluster**, establishing a robust pipeline for generating reference-quality simulation data. Additionally, a **Convolutional Neural Network** was developed and trained on this data, creating a surrogate model capable of near-instantaneous aerodynamic prediction.

The CFD simulations of the NACA airfoil produced lift coefficients that were physically consistent, though

moderately underestimated compared to some experimental data. This behavior is physically consistent with the modeling choices made: the VMS-LES formulation employed in the Navier-Stokes step acts as an implicit filter that dissipates small-scale turbulent structures, suppressing the contribution of fine-grained vortical activity to the overall lift generation. Nonetheless, the results remain well within the range of published numerical and experimental values, confirming the physical validity of the simulation framework by Ladson (1988) and McCroskey (1988).

A further modeling assumption concerns the compressibility of the flow. The simulations were performed at a Mach number significantly below 0.3, a regime in which the relative density variations induced by pressure fluctuations are negligible, making the incompressible Navier-Stokes equations an appropriate model.

Regarding the deep learning implementation, the results demonstrate that a relatively simple Convolutional Neural Network architecture can guarantee robust performance, achieving a Mean Squared Error of 0.0104 on the validation set. The primary advantage of this approach lies in its computational efficiency: while a high-fidelity CFD simulation requires approximately 3 hours on an HPC node, the trained CNN inference time is under one second. Even when accounting for the 5 minutes and 11 seconds required for training, this deep learning framework achieves a remarkable $35.11\times$ speedup compared to conventional methods. This outcome strongly aligns with the premises established by Cuozzo et al. (2026), extending their insights to regular profiles, whereas their approach specifically accounted for deformed airfoils due to ice accretion.

This contrast between computational cost and prediction speed places our work within the context of modern aerodynamic design. From a broader perspective, the high-fidelity approach developed here occupies the opposite end of the cost spectrum from purely surrogate-based methods. For instance, the framework proposed by Cuozzo et al. (2026) achieves a $10^3 - 10^4\times$ reduction in wall-clock time at the cost of a $\sim 14\%$ underestimation of the iced lift curve slope. The present work, instead, prioritizes physical fidelity to provide the reference-quality data that such surrogate models ultimately depend on for training and validation. The two approaches are therefore complementary rather than competing, and the framework developed here constitutes a natural candidate as a high-fidelity data source for future surrogate-based pipelines targeting iced airfoil configurations.

Nevertheless, certain limitations inherent to each method must be acknowledged. These include the computational demand of the RANS simulations and, for the data-driven model, the absence of an *a priori* error estimate, the strict dependency on a training dataset that accurately resembles the target inference domain[20], and the general lack of interpretability characteristic of black-box models.

Looking forward, the framework developed here opens up promising avenues for engineering design and analysis. The speed of the trained CNN makes it ideal for real-time applications and rapid design iterations on consumer-grade hardware, completely bypassing the need for constant HPC access. This capability is particularly relevant for the agile aerodynamic design and field-testing of components for Unmanned Aerial Vehicles and quadcopters, where on-the-fly performance evaluation can accelerate development cycles. The simulation pipeline itself is now poised to serve as a data engine for training future surrogate models on more complex geometries, such as the iced airfoils that motivated this study.

References

P. C. Africa, I. Fumagalli, M. Bucelli, A. Zingaro, M. Fedele, L. Dedè, and A. Quarteroni. lifex-cfd: An open-source computational fluid dynamics solver for cardiovascular applications. *Computer Physics Communications*, 296:109039, 2024.

Ciro Cuozzo, Vincenzo Cusati, Agostino De Marco, Salvatore Corcione, James H. Page, and Alessandro Zanon. Ai-driven surrogate modeling for iced tailplane aerodynamic prediction. *Aerospace Science and Technology*, 168, 2026.

John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(61):2121–2159, 2011. URL `https://jmlr.org/papers/v12/duchi11a.html`.

Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9

[20] As noted, since the training data from the CFD was slightly biased, we expect this bias persists in the surrogate model.

of *Proceedings of Machine Learning Research*, pages 249–256. PMLR, 2010. URL https://proceedings.mlr.press/v9/glorot10a.html.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL https://arxiv.org/abs/1412.6980.

Charles L. Ladson. Effects of independent variation of Mach and Reynolds numbers on the low-speed aerodynamic characteristics of the NACA 0012 airfoil section. Technical Report NASA TM-4074, NASA Langley Research Center, 1988.

W. J. McCroskey. A critical assessment of wind tunnel results for the NACA 0012 airfoil. Technical Report NASA TM-100019, NASA Ames Research Center, 1988.

Philippe R. Spalart and Steven R. Allmaras. A one-equation turbulence model for aerodynamic flows. *AIAA Paper*, 92-0439, 1992.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56): 1929–1958, 2014. URL https://jmlr.org/papers/v15/srivastava14a.html.